

MACHINE LEARNING TUTORIAL

从原理到实践 · 8章+3附录完整手册

# 机器学习 全面教程

覆盖线性回归、决策树、聚类、集成学习  
SVM、神经网络、降维与特征工程

本教程系统讲解机器学习核心算法的原理、流程与实现。每章含计算示例、工程图表与应用实例（房价预测、贷款审批、客户分群、图像压缩等），配套8个Jupyter Notebook与3个数学/框架附录。

8章+3附录

工程图表

8 Notebook

计算示例

应用实例

VERSION v1.0 · 2026.08  
AUDIENCE ML 工程师 · 数据科学家 · 学生  
CONTENT PDF教程 + Notebook代码包  
AUTHOR 博学实验室 (freeLearnLab)

# 目录

|                                     |           |
|-------------------------------------|-----------|
| <b>第一章 机器学习概览</b>                   | <b>5</b>  |
| 1.1 什么是"学": Mitchell 的三要素定义         | 5         |
| 1.2 三大类型: 从"有没有标签"说起                | 5         |
| 1.3 机器学习的标准工作流程                     | 6         |
| 1.4 本教程的脉络                          | 8         |
| 1.5 几个值得关注的现代趋势                     | 8         |
| <b>第二章 线性回归</b>                     | <b>8</b>  |
| 2.1 从问题到模型: 用一条直线描述线性关系             | 8         |
| 2.2 量化"最优": 均方误差 MSE                | 9         |
| 2.3 模型的向量形式表示                       | 9         |
| 2.4 求解最优参数的两条路径                     | 10        |
| 2.5 正规方程: 令梯度为零求解闭式解                | 10        |
| 2.6 梯度下降: 沿损失函数的负梯度方向迭代             | 10        |
| 2.7 正则化: 约束参数空间以抑制过拟合               | 11        |
| 2.8 算法实现: fit / predict / score     | 12        |
| 2.9 手算一个最小例子                        | 13        |
| 2.10 应用实例: 加州房价预测                   | 14        |
| 2.11 代码实现: sklearn LinearRegression | 14        |
| 2.12 现代发展: 线性回归的扩展                  | 15        |
| <b>第三章 决策树</b>                      | <b>15</b> |
| 3.1 问题分解: 选择使子集最纯的特征                | 16        |
| 3.2 熵: 从"不确定性"的数学度量                 | 16        |
| 3.3 信息增益: 分裂导致熵的下降量                 | 16        |
| 3.4 由度量准则推导出算法                      | 17        |
| 3.5 递归分裂的代价: 过拟合风险                  | 18        |
| 3.6 三大经典算法: 同一框架下的三种变体              | 18        |

|                                                    |           |
|----------------------------------------------------|-----------|
| 3.7 算法实现：递归建树 . . . . .                            | 19        |
| 3.8 手算：用信息增益选分裂特征 . . . . .                        | 19        |
| 3.9 应用实例：贷款审批决策树 . . . . .                         | 20        |
| 3.10 代码实现：sklearn DecisionTreeClassifier . . . . . | 21        |
| 3.11 现代发展：从单棵树到森林 . . . . .                        | 21        |
| <b>第四章 聚类分析</b>                                    | <b>22</b> |
| 4.1 聚类问题概述 . . . . .                               | 22        |
| 4.2 K-Means：基于中心的划分式聚类 . . . . .                   | 22        |
| 4.3 层次聚类：基于距离的层级式聚类 . . . . .                      | 25        |
| 4.4 评估指标：无标签条件下的聚类质量度量 . . . . .                   | 25        |
| 4.5 DBSCAN：基于密度的聚类 . . . . .                       | 26        |
| 4.6 算法实现：K-Means 伪代码 . . . . .                     | 30        |
| 4.7 计算示例：5 个一维点的 K-Means . . . . .                 | 30        |
| 4.8 应用实例：客户分群、颜色量化与异常检测 . . . . .                  | 31        |
| 4.9 代码实现：sklearn KMeans 与 DBSCAN . . . . .         | 33        |
| 4.10 现代发展：聚类在深度学习时代 . . . . .                      | 33        |
| <b>第五章 集成学习</b>                                    | <b>35</b> |
| 5.1 三种把模型组合起来的思路 . . . . .                         | 35        |
| 5.2 Bagging 与随机森林：从直觉一步步组装 . . . . .               | 36        |
| 5.3 Boosting：让后面的模型专攻前面的错题 . . . . .               | 37        |
| 5.4 XGBoost 与 LightGBM：从"GBDT 太慢"到工程优化 . . . . .   | 37        |
| 5.5 算法实现描述：随机森林的伪代码 . . . . .                      | 38        |
| 5.6 计算示例：手算一次 Bagging 投票 . . . . .                 | 39        |
| 5.7 应用实例：特征重要性分析 . . . . .                         | 39        |
| 5.8 代码实现：sklearn 与 XGBoost . . . . .               | 40        |
| 5.9 现代发展与小结 . . . . .                              | 40        |
| <b>第六章 支持向量机</b>                                   | <b>42</b> |
| 6.1 从直觉到优化问题：一步步推导 . . . . .                       | 42        |
| 6.2 软间隔：从约束到 hinge loss . . . . .                  | 43        |

|                                               |           |
|-----------------------------------------------|-----------|
| 6.3 梯度推导：从 hinge loss 到更新规则 . . . . .         | 43        |
| 6.4 核函数：当数据线性不可分时 . . . . .                   | 44        |
| 6.7 算法实现描述 . . . . .                          | 45        |
| 6.8 计算示例：手算 2D SVM 间隔 . . . . .               | 46        |
| 6.9 应用实例：手写数字识别 . . . . .                     | 46        |
| 6.10 代码实现：sklearn SVC . . . . .               | 46        |
| 6.11 现代发展与小结 . . . . .                        | 47        |
| <b>第七章 神经网络基础</b>                             | <b>48</b> |
| 7.1 神经元：加权投票 + 决策 . . . . .                   | 48        |
| 7.2 把单个神经元串起来：多层感知机 . . . . .                 | 48        |
| 7.3 怎么训练：前向传播 + 反向传播 . . . . .                | 49        |
| 7.4 反向传播的链式推导：从输出层一路倒推回来 . . . . .            | 50        |
| 7.5 激活函数：没有非线性，多层等于一层 . . . . .               | 51        |
| 7.6 为什么 ReLU 比 Sigmoid 好：梯度消失问题 . . . . .     | 51        |
| 7.7 损失函数与优化器 . . . . .                        | 52        |
| 7.8 算法实现描述 . . . . .                          | 52        |
| 7.9 计算示例：2 层网络的前向 + 反向手算 . . . . .            | 53        |
| 7.10 应用实例：MNIST 手写识别 . . . . .                | 53        |
| 7.11 代码实现：PyTorch MLP . . . . .               | 54        |
| 7.12 现代发展与小结 . . . . .                        | 55        |
| <b>第八章 降维与特征工程</b>                            | <b>57</b> |
| 8.1 PCA 的直觉：找数据变化最大的方向 . . . . .              | 57        |
| 8.2 PCA 的数学推导：协方差矩阵与特征值分解 . . . . .           | 58        |
| 8.3 算法实现描述：PCA 的伪代码 . . . . .                 | 58        |
| 8.4 计算示例：2D $\rightarrow$ 1D PCA 手算 . . . . . | 59        |
| 8.5 t-SNE：当 PCA 找不到非线性结构时 . . . . .           | 59        |
| 8.6 特征工程：再好的算法也救不了烂数据 . . . . .               | 60        |
| 8.7 应用实例：高维数据可视化 . . . . .                    | 61        |
| 8.8 代码实现：sklearn PCA 与 t-SNE . . . . .        | 61        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 8.9 现代发展与小结 . . . . .                               | 62        |
| <b>附录 A 数学基础</b>                                    | <b>63</b> |
| A.1 线性代数：向量是数据，矩阵是变换，特征值是主方向 . . . . .              | 63        |
| A.2 概率论：贝叶斯是"用新证据更新旧信念" . . . . .                   | 63        |
| A.3 微积分：梯度是"变化最快的方向" . . . . .                      | 64        |
| A.4 信息论：交叉熵是分类损失的"标配" . . . . .                     | 65        |
| <b>附录 B 集成学习框架</b>                                  | <b>66</b> |
| B.1 Scikit-learn：通用入门的首选 . . . . .                  | 66        |
| B.2 XGBoost：Kaggle 表格任务的常胜将军 . . . . .              | 66        |
| B.3 LightGBM：大数据上的速度冠军 . . . . .                    | 66        |
| B.4 CatBoost：类别特征的原生支持者 . . . . .                   | 67        |
| B.5 框架对比与选型经验 . . . . .                             | 67        |
| <b>附录 C PyTorch 与 Transformers</b>                  | <b>68</b> |
| C.1 PyTorch 基础：Tensor、autograd、Module 三件套 . . . . . | 68        |
| C.2 数据加载：Dataset 与 DataLoader . . . . .             | 68        |
| C.3 HuggingFace Transformers：让大模型触手可及 . . . . .     | 69        |
| C.4 代码示例：BERT 文本分类微调 . . . . .                      | 69        |
| C.5 进阶话题与小结 . . . . .                               | 70        |

# 第一章 机器学习概览

假设你手上有一家电商的 10 万条用户购买记录，业务方要求预测哪些用户下个月会流失。传统做法是人工写规则——“近 30 天未登录”“最近一次购物超过 60 天”“客单价低于 50 元”等。然而规则数量随业务复杂度线性增长，当规则累积到 100 条以上时，规则间的相互冲突、时变失效以及归因困难将使该规则体系难以维护。

机器学习提供了一种范式转换——不再依赖人工编写的规则，而是让算法从历史数据中归纳出模式。具体而言，将带“是否流失”标签的历史记录作为输入，算法自动学习从特征到标签的映射，进而对新用户输出流失概率。由此，决策规则不再由人工显式编码，而是从数据中估计而得。

这一思路回应了一个根本问题：何谓“学习”。本章先厘清该定义，再介绍机器学习的三大类型，最后给出标准工作流程。

## 1.1 什么是“学”：Mitchell 的三要素定义

卡内基梅隆大学的 Tom Mitchell 给出了一个被广泛接受的定义：如果一个程序在某个任务 **T** 上的性能 **P**，会随着经验 **E** 的增加而提升，则称该程序从经验 **E** 中“学”到了知识。经验 **E**、任务 **T**、性能 **P** 三者构成理解所有机器学习算法的基本框架。

回到前述流失预测示例：经验 **E** 即为带标签的 10 万条历史用户记录；任务 **T** 是“对给定新用户，判定其下月是否流失”；性能 **P** 由准确率、AUC 等指标度量。当训练样本数单调增加时，模型预测性能同步提升——这就是“学习”在数学上的精确含义。

该范式与传统“人工编写规则”的关键差异在于：**知识不是显式编码的，而是从数据中归纳而得**。机器学习近十年的快速发展依赖三方面合力：其一，互联网使标注数据获取成本大幅下降；其二，GPU 使大规模矩阵运算成为日常算力；其三，从逻辑回归到 Transformer 的算法演进持续扩展模型表达能力。三者中缺一，机器学习都难以达到当前规模。

## 1.2 三大类型：从“有没有标签”说起

机器学习通常分为三大类：监督学习、无监督学习、强化学习。三者的核心区分在于训练数据的形态：**训练数据中是否含有标签**。

若每条训练数据均给出正确答案（如流失表中每行均标注“流失/未流失”），则算法在标签的“监督”下进行学习，称为**监督学习**。其代表任务有两类：预测标签为离散类别（如流失/不流失、垃圾邮件/正常邮件）的称为**分类**；预测标签为连续数值（如房价、销量）的称为**回归**。线性回归、逻辑回归、决策树、SVM、神经网络均属此类。

若数据仅含特征而无标签（如电商 100 万用户的行为日志，未事先标注用户类型），则训练过程缺乏监督信号，称为**无监督学习**。此时算法需自行发现数据结构：将相似用户聚为同簇（聚类）、将高维数据投影到低维以观察分布（降维）、或挖掘“购买 A 的用户通常也购买 B”等关联规则。K-Means、DBSCAN、PCA 均属此类。

强化学习与前两者形态不同：不存在静态数据集，而存在一个可反馈的环境。智能体每执行一个动作，环境即返回一个奖励或惩罚信号；其目标是学习一个策略，使长期累积奖励的期望最大化。AlphaGo、机器人控制、推荐系统中的 RLHF 均属此类。本教程不展开强化学习，读者仅需了解其基本思想即可。



图 1.1 机器学习三大类型对比：监督学习有标签，无监督学习无标签，强化学习依赖奖励信号

**[提示]** 实际中三类边界正在模糊化。半监督学习仅使用少量带标签样本配合大量无标签样本；自监督学习（BERT、GPT 预训练）利用数据自身结构构造监督信号，本质是"自我监督"。但作为入门框架，"是否有标签"这一划分仍是当前最实用的认知坐标。

## 1.3 机器学习的标准工作流程

机器学习项目遵循一条从问题理解到模型运维的标准化流程。该流程不是经验性的产物，而是从"数据驱动决策"这一方法论出发自然形成的工程范式。完整流程由七个阶段构成：业务理解、数据收集、数据预处理、特征工程、模型训练、模型评估、部署与监控。各阶段依次推进，前一阶段输出作为后一阶段输入，形成闭环。

### 1.3.1 业务理解：将业务问题形式化为数学问题

第一阶段的目标是将业务需求转化为可量化、可优化的数学问题。具体工作包括：明确预测目标（即标签定义）、确定可用特征空间、定义性能度量（准确率、召回率还是收益）、明确工程约束（延迟、可解释性、公平性等）。该阶段决定后续所有工作的方向；若目标定义偏离业务本质，后续投入将无法产生对应价值。

### 1.3.2 数据收集：构造与问题匹配的样本集

第二阶段需根据形式化后的数学问题收集对应数据。常见数据源包括关系数据库、API 接口、应用日志、网络爬虫与人工标注。此阶段需评估样本量是否充足、覆盖是否均衡、标签噪声水平以及特征缺失比例。数据质量决定了模型性能的上限——任何算法都难以从有偏或低质数据中归纳出稳健的规律。

### 1.3.3 数据预处理：规范化原始数据



第三阶段对原始数据做规范化处理。常见操作包括：缺失值填补（均值、中位数或模型预测）、异常值剔除（基于分位数或孤立森林）、重复样本去重、字段类型统一、量纲对齐。机器学习工程实践有一条铁律——"Garbage in, garbage out"：输入数据的质量直接决定输出模型的质量。

### 1.3.4 特征工程：将原始字段转化为模型可用表示

第四阶段将原始字段加工为模型可消化的数值表示。典型操作包括：数值归一化（StandardScaler、MinMaxScaler）、类别编码（One-Hot、Target Encoding）、时间特征提取（周期项、趋势项）、交叉特征构造以及特征选择。特征工程的质量常决定模型性能的上限——在传统机器学习场景中尤其如此。

### 1.3.5 模型训练：选择算法并估计参数

第五阶段选择模型算法并估计其参数。具体步骤包括：选择模型族（线性回归、决策树、神经网络等）、划分训练集与测试集（通常按 8:2 或 7:3）、配置超参数、进行交叉验证。需特别注意**数据泄漏**问题：训练阶段不得使用任何来自测试集的信息，否则评估结果将无法反映模型的真实泛化能力。

### 1.3.6 模型评估：估计泛化性能

第六阶段在**未参与训练的测试集**上评估模型。分类问题常用准确率、精确率、召回率、F1、AUC；回归问题常用 MSE、MAE、 $R^2$ ；聚类问题常用轮廓系数、Davies-Bouldin 指数。关键原则是：评估指标必须基于独立测试集；用训练集指标评估自身属于方法论错误。

### 1.3.7 部署与监控：模型上线与持续运维

第七阶段将模型部署至生产环境并持续监控。真实世界的分布会随时间发生**数据漂移**（data drift），因此必须持续监控预测分布、特征分布与性能衰减，并建立再训练触发机制、版本管理与灰度发布流程。工业实践中，部署监控常占项目总工作量的 60%-80%，远超模型训练本身。

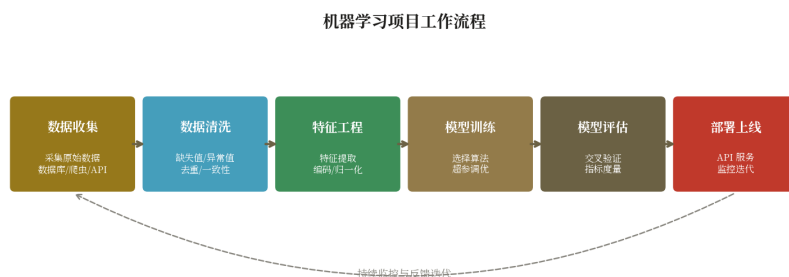


图 1.2 机器学习标准工作流程：业务理解→数据收集→数据预处理→特征工程→模型训练→模型评估→部署监控的七阶段闭环

**[技巧]** 在工业实践中，数据收集、数据预处理与特征工程合计通常占项目总时长的 60%-80%，模型训练仅占 10%-20%。工程师的经验法则是“先看数据，再看模型”——特征工程的质量常决定最终性能上限。



## 1.4 本教程的脉络

后续三章按"由浅入深、由监督到无监督"的顺序介绍三个经典算法。第二章**线性回归**——最简单也最具启发性的监督学习模型，从中可自然推出损失函数、梯度下降、正则化三个贯穿全书的核心概念。

第三章**决策树**——最接近人类决策模式的可解释模型。从中可见"熵""信息增益"等概念如何从"将数据划分得更纯净"这一数学目标逐步推导而得，进而引出剪枝与集成学习的基础。

第四章**聚类分析**——进入无监督学习领域。将依次介绍基于中心的划分式聚类（K-Means）、基于距离的层级式聚类（层次聚类）、以及基于密度的聚类（DBSCAN）三类不同思路，最后给出统一的评估指标。

这三章构成后续内容的地基。SVM、神经网络、深度学习、强化学习均建立在这些基本概念之上——再复杂的 Transformer，本质上也仍在监督或自监督框架下求解相应问题。

## 1.5 几个值得关注的现代趋势

在动手学习具体算法之前，先简要梳理机器学习领域的几个现代发展方向，以便后续章节出现相关术语时不致混淆。

- **AutoML**：将特征工程、模型选择、超参调优全自动化。代表工具有 Auto-sklearn、TPOT、H2O AutoML，降低 ML 使用门槛。
- **MLOps**：将 DevOps 思路引入 ML 生命周期，解决模型版本管理、流水线编排、漂移监控、灰度发布。代表工具：MLflow、Kubeflow、Weights & Biases。
- **大模型时代**：GPT、Claude、Gemini 等大语言模型正在重塑 ML 范式。"预训练+微调（含 LoRA）+ RLHF"成为新标准，"模型即服务"成为常态。
- **自监督学习**：利用数据自身结构作为监督信号（如掩码语言建模），大幅减少对人工标注的依赖，是 BERT、ViT、CLIP 成功的关键。
- **负责任 AI**：公平性、可解释性、隐私保护、稳健性正在成为工业级系统的硬性指标，欧盟的 AI Act 已落地。

上述趋势背后所依赖的依然是本章介绍的工作流程与三大类型划分。本章是后续所有内容的方法论基础。

# 第二章 线性回归

给定一组面积与房价数据——如 50 平对应 80 万、80 平对应 130 万、120 平对应 200 万——现要求对一套 100 平的房屋估价。这是本章要解决的问题。问题虽简单，却蕴含机器学习几乎所有核心思想：损失函数、梯度下降、正则化均可从此例逐步推出。

## 2.1 从问题到模型：用一条直线描述线性关系

面对这组面积-价格数据，第一步是可视化。将每个房屋绘制为坐标点，横轴为面积、纵轴为价格，可得散点图。从图中可观察到面积与价格大致呈正比，样本点近似沿一条直线分布。

因此，对 100 平房屋的估价，可归结为构造一条穿过样本点的直线，再读取该直线在  $x=100$  处的纵坐标。

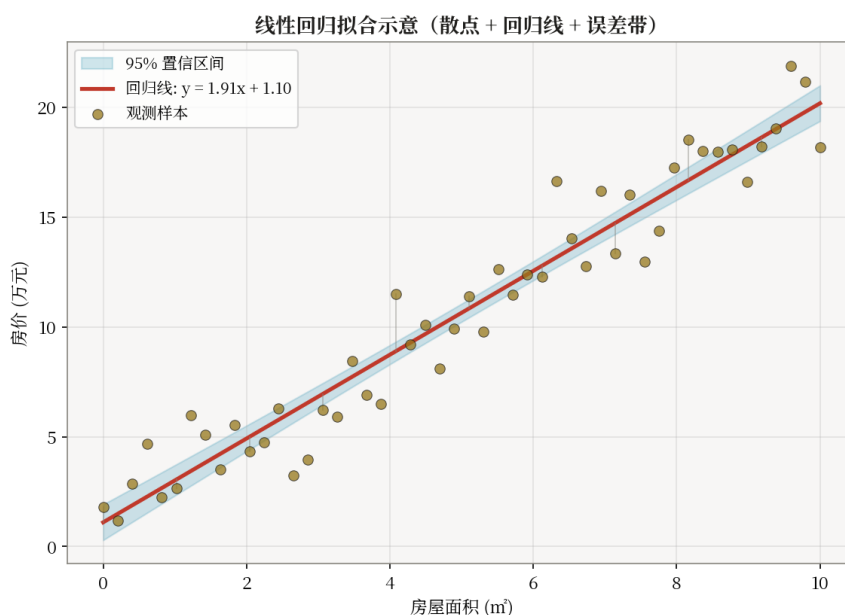


图 2.1 散点图与拟合直线：面积越大价格越高，一条直线即可近似描述该关系

然而穿过样本点的直线有无穷多条，何为“最优”？部分直线压在样本上方、部分位于样本下方、部分仅穿过少数点。因此需对“最优”给出严格的数学定义。这是机器学习中的常见模式——先明确优化目标，再设计达成目标的算法。

## 2.2 量化“最优”：均方误差 MSE

何为“好”的拟合直线？最直接的判据是：该直线对已有样本的预测误差整体最小。具体而言，对每个真实点计算其与直线的偏差——若真实值为 80 万而预测值为 85 万，则偏差为 5 万；若真实值为 200 万而预测值为 195 万，偏差同样为 5 万。将所有偏差汇总，使其总体最小即为目标。

然而偏差有正有负，直接求和将发生正负抵消。因此需将偏差做非线性变换以消除符号影响。一种自然的变换是平方——既保证非负，又对大偏差施加更重惩罚（平方放大效应）。进而除以样本数取平均，即得均方误差（Mean Squared Error, MSE）：

$$L(w, b) = (1/n) \cdot \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

其中  $\hat{y}_i$  为模型对第  $i$  个样本的预测值， $y_i$  为真实值。MSE 是一种**损失函数**——损失函数即用于度量模型预测与真实值之间偏差的标量函数，其值越小，模型越优。由此，问题被形式化为：寻找一组参数  $(w, b)$  使得 MSE 最小。

## 2.3 模型的向量形式表示

将上述直觉翻译为数学公式。一条直线可写作  $y = wx + b$ ，其中  $w$  为斜率（每单位面积对应的价格增量）， $b$  为截距（面积为 0 时的“地价”基线）。若存在多个特征——面积、卧室数、房龄共同决定价格——则推广为多元线性形式：

$$\hat{y}_i = w_1 \cdot x_{i,1} + w_2 \cdot x_{i,2} + \dots + w_d \cdot x_{i,d} + b$$

采用向量表示更为简洁： $\hat{y} = Xw + b$ 。其中  $X$  是将所有样本的特征排列而成的矩阵（ $n$  行  $d$  列）， $w$  是  $d$  维权重向量， $b$  是标量偏置。该公式即线性回归模型的完整数学定义——所谓模型，即一组参数  $(w, b)$  加上该公式。训练过程即是寻找使 MSE 最小的  $(w, b)$ 。

## 2.4 求解最优参数的两条路径

至此问题已转化为纯优化问题：在 MSE 这一凸曲面上寻找最低点。该最低点对应的  $(w, b)$  即为最优参数。求解该优化问题有两条路径：其一是**闭式解**（解析法），即直接通过求导令导数为零求解；其二是**迭代法**，即从初始点出发沿损失下降方向逐步逼近最优。

闭式解的依据是：MSE 为参数  $(w, b)$  的二次凸函数，其最小值点处梯度为零。因此可将梯度表达式令为零，求解线性方程组即得最优参数。该路径推出**正规方程**。迭代法的依据是：在可微凸函数上，沿负梯度方向更新参数可使函数值单调下降。该路径推出**梯度下降**。

两条路径各有适用场景。闭式解一步到位，但当样本数  $n$  或特征维度  $d$  较大时，矩阵求逆的计算复杂度  $O(d^3)$  不可承受；迭代法收敛较慢，但可处理任意规模数据，是深度学习的训练基础。下文分别介绍二者。

## 2.5 正规方程：令梯度为零求解闭式解

首先将 MSE 写成矩阵形式。为简化推导，将偏置  $b$  吸收进  $w$  中——做法是在  $X$  的最左侧补一列全 1，使  $w$  多出一个分量作为  $b$ 。于是 MSE 可写为  $L(w) = (1/n) \cdot \|Xw - y\|^2$ ，展开后得  $L(w) = (1/n) \cdot (Xw - y)^T (Xw - y)$ 。

对  $w$  求梯度并令其为零，展开求导过程如下：

$$\partial L / \partial w = (2/n) \cdot X^T (Xw - y) = 0$$

由该等式出发，移项化简即得**正规方程**：

$$X^T X \cdot w = X^T y$$

若  $X^T X$  可逆（即特征之间不存在完全线性相关），则两边同时左乘其逆矩阵得：

$$w^* = (X^T X)^{-1} X^T y$$

这就是**最小二乘解**——“最小二乘”之名来自“使误差平方和最小”。该解为一步到位的解析表达式，仅需一次矩阵运算即可得结果。然而其代价是  $X^T X$  求逆，复杂度为  $O(d^3)$ 。当  $d$  为数百至数千时尚可承受，若  $d$  达到数十万量级（如文本词向量维度），矩阵求逆将不再可行。由此需引入迭代法——梯度下降。

## 2.6 梯度下降：沿损失函数的负梯度方向迭代

当样本数  $n$  或特征维度  $d$  较大时，正规方程中  $X^T X$  求逆的代价过高，闭式解不再可行。因此需要一种基于迭代的优化方法——梯度下降法。

梯度下降的数学依据是一个凸优化基本事实：对于可微函数  $L(w)$ ，其梯度  $\nabla L(w)$  指向函数值上升最快的方向；因此沿负梯度方向更新参数，可使函数值以最快速率下降。当  $L$  为凸函数时，该迭代过程保证收敛到全局最优；当  $L$  为非凸函数时，则仅保证收敛到局部最优。

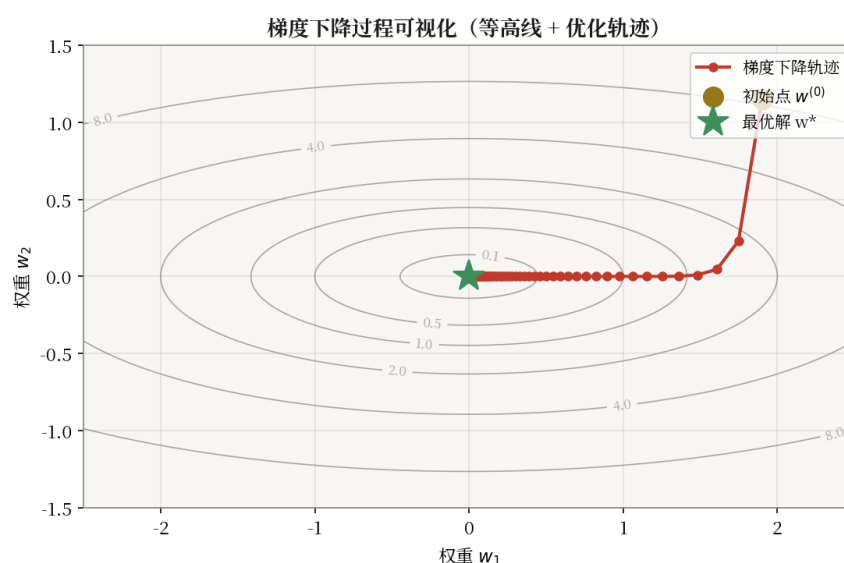


图 2.2 梯度下降示意：从初始点出发，沿损失曲面负梯度方向迭代逼近最小值点

将上述原理应用于线性回归。损失函数为  $L(w, b) = (1/n) \cdot \sum (\hat{y}_i - y_i)^2$ ，其中  $\hat{y}_i = w \cdot x_i + b$ 。对  $w$  与  $b$  分别求偏导：

$$\partial L / \partial w = (2/n) \cdot \sum_i (\hat{y}_i - y_i) \cdot x_i, \quad \partial L / \partial b = (2/n) \cdot \sum_i (\hat{y}_i - y_i)$$

由此得到参数更新规则——沿负梯度方向以步长  $\eta$  更新：

$$w \leftarrow w - \eta \cdot \partial L / \partial w, \quad b \leftarrow b - \eta \cdot \partial L / \partial b$$

其中  $\eta$  称为**学习率** (learning rate)，用于控制每步的更新幅度。上述偏导写成矩阵形式更为简洁： $\nabla_w = (2/n) \cdot X^T(Xw - y)$ ， $\nabla_b = (2/n) \cdot \sum(Xw - y)$ 。将两梯度代入更新公式反复迭代，损失函数将单调下降，最终稳定于某个极小值点。

学习率  $\eta$  是关键超参数，需谨慎选择。若  $\eta$  过大，单步更新将跨越极小值点而发散，损失反而上升；若  $\eta$  过小，每步下降幅度极微，收敛将异常缓慢。实践中常从 0.01、0.001 等小值起步，配合学习率衰减策略（如余弦退火）或自适应优化器（如 Adam）使用，以兼顾收敛速度与稳定性。

**[提示]** 当样本数  $n$  较大时，每步使用全部  $n$  个样本计算梯度代价过高。此时可采用**随机梯度下降 SGD**（每步仅抽 1 个样本）或**小批量 SGD**（每步抽 32/64/128 个样本）。小批量 SGD 是深度学习训练的事实标准——既保证梯度估计的方差可控，又支持 GPU 并行加速。

## 2.7 正则化：约束参数空间以抑制过拟合

至此线性回归看似完备。然而在实际应用中常出现以下现象：当特征维度  $d$  远大于样本量  $n$ （如  $d=100, n=30$ ）时，模型在训练集上误差接近 0，但泛化到新数据时性能急剧下降。该现

象称为**过拟合**——模型过度拟合训练样本的特定噪声，而非数据背后的普遍规律。

过拟合的成因是：当  $d > n$  时，参数空间自由度过高，最小二乘解可使训练集 MSE 严格等于零，但代价是参数取值过大且对噪声高度敏感，致使新样本稍有偏移即产生大幅预测偏差。由此，抑制过拟合的直接方法是对参数空间施加约束，限制参数幅度。具体做法是在损失函数中增加**惩罚项**，使参数自身的范数也被计入代价。

最常用的两种惩罚形式。其一是对参数平方和的惩罚，称为 **L2 正则化**（亦称岭回归 Ridge）：

$$L_{\text{Ridge}} = \text{MSE} + \lambda \cdot \sum_j w_j^2$$

L2 正则化的效果是将所有权重整体压小，但通常不至于压到精确为零，因此适合“所有特征都有一定贡献”的场景。其二是对参数绝对值和的惩罚，称为 **L1 正则化**（亦称 Lasso）：

$$L_{\text{Lasso}} = \text{MSE} + \lambda \cdot \sum_j |w_j|$$

L1 正则化的几何约束区域为菱形，与损失等高线最易在顶点处相切——而顶点恰好意味着某些  $w_j = 0$ 。因此 Lasso 可将不重要特征的权重精确压至零，起到**特征选择**的作用，适合高维稀疏数据（如基因表达、文本词频）。

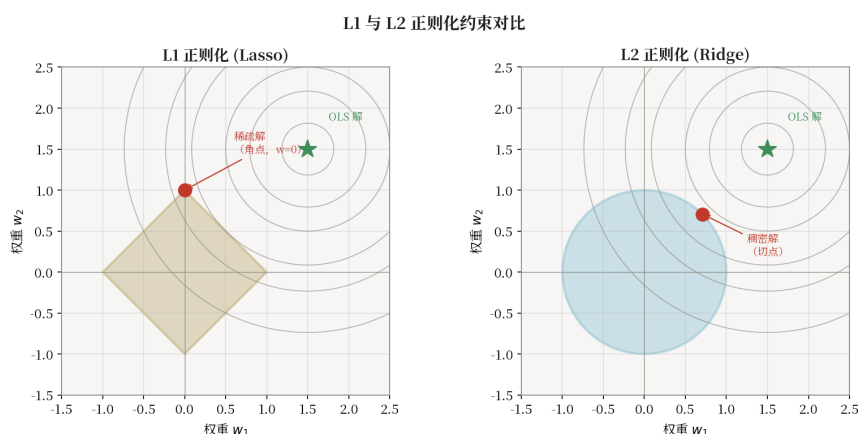


图 2.3 L1 与 L2 正则化几何对比：L1 约束区为菱形，易在顶点相切产生稀疏解

$\lambda$  为正则化强度超参数。 $\lambda=0$  时退化为普通最小二乘； $\lambda$  过大则模型过于保守而出现欠拟合。实际中  $\lambda$  通常通过交叉验证选择。若同时需要 Lasso 的稀疏性与 Ridge 的稳定性，可采用 **ElasticNet**——将两者按比例混合。

## 2.8 算法实现：fit / predict / score

将上述推导收拢为代码实现，线性回归的核心即三个方法：**fit**（训练，求出  $w$  和  $b$ ）、**predict**（预测，将新  $X$  代入  $\hat{y} = Xw + b$ ）、**score**（评估，计算  $R^2$ ）。其伪代码结构如下。

### 2.8.1 数据结构

- **权重向量  $w$** ：长度  $d$ （特征数），是模型的核心参数
- **偏置  $b$** ：标量
- **训练数据**： $X \in \mathbb{R}^{n \times d}$ ,  $y \in \mathbb{R}^n$

## 2.8.2 fit(X, y) — 梯度下降版本

输入：特征矩阵  $X$ ，标签向量  $y$ ，学习率  $\alpha$ ，迭代次数  $T$

过程：

- 1. 初始化  $w \leftarrow 0, b \leftarrow 0$
- 2. 对  $\text{epoch} = 1, 2, \dots, T$ :
  - (a) 算预测值  $\hat{y} = Xw + b$
  - (b) 算误差  $\delta = \hat{y} - y$
  - (c) 算梯度：  $\nabla_w = (2/n) \cdot X^T \delta, \nabla_b = (2/n) \cdot \Sigma \delta$
  - (d) 更新参数：  $w \leftarrow w - \alpha \cdot \nabla_w, b \leftarrow b - \alpha \cdot \nabla_b$

替代方案（闭式解）：直接计算  $w^* = (X^T X)^{-1} X^T y$ ，一步求解

## 2.8.3 predict(X) 与 score(X, y)

$$\hat{y} = Xw + b$$

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

$R^2$  度量模型相对于“以均值作为预测”的改进程度。 $R^2=1$  表示完美预测， $R^2=0$  表示与均值预测等同， $R^2<0$  则劣于均值预测。

## 2.8.4 复杂度

- 闭式解：  $O(nd^2 + d^3)$ ，矩阵求逆是瓶颈
- 梯度下降：  $O(ndT)$ ， $T$  为迭代次数
- 预测：  $O(nd)$ ，单次矩阵向量乘法

**[提示]** scikit-learn 的 LinearRegression 采用 SVD 分解替代直接求逆，数值更稳定，能处理  $X^T X$  不可逆的情形。

## 2.9 手算一个最小例子

为验证上述推导，以三个样本手动演算。已知三个点  $(x, y) = \{(1, 2), (2, 4), (3, 5)\}$ ，用最小二乘法拟合  $y = wx + b$ 。

解：先计算所需统计量——

$$\Sigma x = 1+2+3 = 6, \Sigma y = 2+4+5 = 11, \Sigma x^2 = 1+4+9 = 14, \Sigma xy = 2+8+15 = 25$$

样本数  $n = 3$ ，均值  $\bar{x} = 2, \bar{y} = 11/3$

代入闭式解公式：

$$w = (n \cdot \Sigma xy - \Sigma x \cdot \Sigma y) / (n \cdot \Sigma x^2 - (\Sigma x)^2) = (3 \times 25 - 6 \times 11) / (3 \times 14 - 36) = 9/6 = 1.5$$

$$b = \bar{y} - w \cdot \bar{x} = 11/3 - 1.5 \times 2 = 11/3 - 3 = 2/3 \approx 0.667$$



由此拟合直线为  $y = 1.5x + 0.667$ 。代入  $x=1,2,3$  验证： $\hat{y} \approx 2.17, 3.67, 5.17$ ，残差平方和  $SSE \approx 0.167$ ，已是所有直线中的最小值。

该示例虽仅含 3 个点，却完整演示了"定义损失→求导→令导数为零→求解参数"的全过程。真实数据的处理无非是  $n$  大几个数量级，方法论完全一致。

## 2.10 应用实例：加州房价预测

加州房价数据集 (California Housing) 是 sklearn 内置的经典回归数据集，包含 20640 个街区样本，8 个特征：MedInc (地区收入中位数)、HouseAge (房龄)、AveRooms (人均房间数)、AveBedrms (人均卧室数)、Population (人口)、AveOccup (人均居住人数)、Latitude (纬度)、Longitude (经度)，目标为 MedHouseVal (街区房价中位数，单位 10 万美元)。本例以线性回归建立基线，再用 Ridge 与 Lasso 对比。

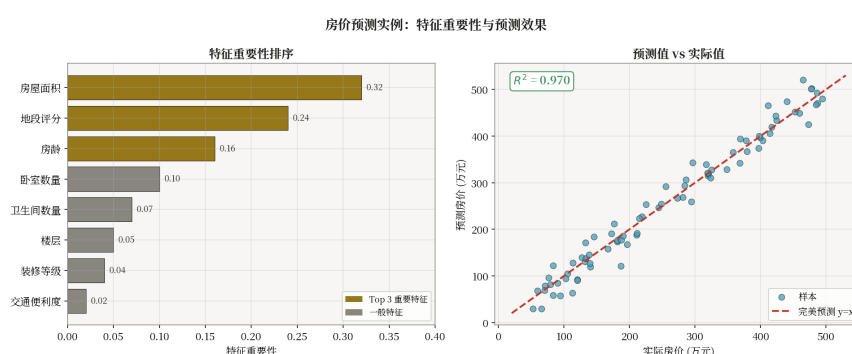


图 2.4 加州房价预测：真实值 vs 预测值散点（左）与各特征权重条形图（右）

基线 LinearRegression 测试集  $R^2 \approx 0.60$ ; Ridge ( $\lambda=1.0$ )  $R^2 \approx 0.60$ ，差异不大说明该数据集特征共线性较弱；Lasso 可将部分弱相关特征的权重压至零，起到特征筛选作用。若需进一步提升精度，可加入特征交叉（如人均房间数 = AveRooms / AveOccup）、地理聚类特征，或直接换用梯度提升树 (GBDT) 等更强模型。然而线性回归的真正价值不在绝对精度，而在于它是所有更强模型的**基线**，更在于其可解释性：每个特征的权重直接给出"面积每多 1 平，房价平均增量"。

## 2.11 代码实现：sklearn LinearRegression

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# -- 加载加州房价数据集 --
X, y = fetch_california_housing(return_X_y=True)
print(f"样本数: {X.shape[0]}, 特征数: {X.shape[1]}")

# -- 划分训练集与测试集 (8:2, 固定随机种子) --
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
```



```
# -- 特征标准化（必须用训练集统计量 fit，避免数据泄漏） --
scaler = StandardScaler().fit(X_tr)
X_tr_s = scaler.transform(X_tr)
X_te_s = scaler.transform(X_te)

# -- 训练三个模型：普通 / Ridge / Lasso --
models = {
    "LinearRegression": LinearRegression(),
    "Ridge(a=1.0)": Ridge(alpha=1.0),
    "Lasso(a=0.1)": Lasso(alpha=0.1),
}
for name, m in models.items():
    m.fit(X_tr_s, y_tr)
    y_pred = m.predict(X_te_s)
    rmse = np.sqrt(mean_squared_error(y_te, y_pred))
    r2 = r2_score(y_te, y_pred)
    print(f"{name:20s} RMSE={rmse:.3f}, R2={r2:.3f}")

# -- 查看 Lasso 权重（部分会被压为 0） --
print("Lasso 权重:", models["Lasso(a=0.1)"].coef_)
```

## 2.12 现代发展：线性回归的扩展

线性回归形式简洁，但其思想贯穿整个机器学习发展史。现代许多复杂模型在数学结构上可视为“线性回归 + 各种扩展”。

- **广义线性模型 GLM**：将线性回归扩展到分类（逻辑回归）、泊松回归、Gamma 回归等，通过链接函数  $g(\mu) = Xw$  连接线性预测与分布参数。
- **自动化特征工程**：Featuretools、AutoFeat 等工具自动构造交叉、聚合、时序特征，降低手工成本。
- **自动化超参调优**：Optuna、Hyperopt、GridSearchCV 等，结合贝叶斯优化、Hyperband 实现高效调参。
- **正则化进化**：ElasticNet（L1+L2 混合）、Group Lasso（组稀疏）、Fused Lasso（结构稀疏）满足不同约束需求。
- **可解释性方法**：SHAP、LIME 将任意黑箱模型“翻译”成线性加权形式，弥合精度与可解释性的鸿沟。

本章看似简单，实为后续内容的方法论基础。逻辑回归、神经网络、SVM 等后续模型，本质上均在“加权求和”这一基本操作上做文章——区别仅在于如何放松“线性”这一假设。

## 第三章 决策树

银行每日处理大量贷款申请：是否批准某位申请人的放贷请求？传统做法是请风控专家编写一组 if-else 规则——“月薪 > 1 万 且 征信 > 700 才批准”“负债率 > 50% 直接拒绝”等。然而规则数量随业务复杂度增长后，规则间相互冲突、时变失效、难以归因等问题将凸显。更根本的限制在于：能编写规则的专家数量有限。由此引出一个问题：能否让算法从历史数据中自动归纳出决策规则？

这正是决策树（Decision Tree）要解决的问题。决策树从训练数据中自动归纳出 if-else 规则，其结构为一棵倒置的树——每条从根到叶的路径对应一条人可读的决策规则。相比线性回

归的全局线性假设，决策树可天然处理非线性关系与特征交互，同时保持白盒可解释性。这是其在金融风控、医疗诊断、营销响应等领域长期占据主导地位的根本原因。

### 3.1 问题分解：选择使子集最纯的特征

面对一份历史贷款数据（如 6 条记录，每条标注"批准/拒绝"），算法的第一步是：**选择一个最能区分类别结果的特征进行分裂**。何为"最能区分"？最直接的判据是分裂后子节点中样本应尽量属于同一类别，而非混合。

以"有工作"这一特征为例。若数据中"有工作"的样本几乎全部批准、"无工作"的几乎全部拒绝，则按该特征分裂后，左子节点全为 Y、右子节点全为 N，称为"纯分裂"。反之，"信用好"分裂后左子节点仍 Y/N 各半、右子节点也 Y/N 各半，则该分裂未提供有效信息。

然而"纯"只是口语描述，算法需要将其**量化**——否则无法比较两个特征的分裂效果，也无法设定停止阈值。由此引出本章的核心概念：**熵与信息增益**。

### 3.2 熵：从"不确定性"的数学度量

要量化"分裂后子节点的纯度"，需先量化"一组数据的不确定性程度"。这正是**熵**（Entropy）所度量的对象。其数学动机如下：若一组样本全属同一类，则其类别完全确定，熵应为零；若两类各占一半，则下一样本的类别最难预测，熵应取最大值。

满足上述性质的数学形式即为香农熵：

$$H(D) = - \sum_{k=1}^K p_k \cdot \log_2 p_k$$

其中  $p_k$  是第  $k$  类样本在  $D$  中的占比。公式可拆解为三部分： $\log_2 p_k$  度量"该类出现所带来的信息量"（小概率事件信息量大）；乘以  $p_k$  实现"按出现频率加权"；前置负号则保证结果非负（因  $p_k < 1$  时  $\log$  为负）。

代入两个极端情形验证。全 Y 情形： $p_Y = 1, p_N = 0, H = -(1 \cdot \log 1 + 0 \cdot \log 0) = 0$  bit——完全确定，熵为零。Y/N 各半情形： $p_Y = p_N = 0.5, H = -(0.5 \cdot \log 0.5 + 0.5 \cdot \log 0.5) = 1$  bit——最难预测，熵最大。由此得到基本判据：**熵越低越纯，熵越高越混乱**。

### 3.3 信息增益：分裂导致熵的下降量

有了熵的定义，"分裂后是否更纯"即可量化为**分裂前后熵的下降量**。下降量越大，说明该次分裂降低不确定性的程度越大，对应特征越有价值。该差值定义为**信息增益**。

设特征  $A$  将  $D$  分为  $v$  个子集  $D_1, D_2, \dots, D_v$ ，每个子集有自身的熵  $H(D_v)$ 。将这些子集的熵按子集大小加权平均，得到分裂后的"平均熵"，即**条件熵**  $H(D|A)$ ：

$$H(D|A) = \sum_v (|D_v| / |D|) \cdot H(D_v)$$

进而，信息增益为分裂前的熵减去分裂后的条件熵：

$$g(D, A) = H(D) - H(D|A)$$

由此， $g(D, A)$  即"使用特征 A 后，数据不确定性下降的程度"。下降越多，A 越值得选用。这是 ID3 算法的核心准则：每一步均选择信息增益最大的特征进行分裂。

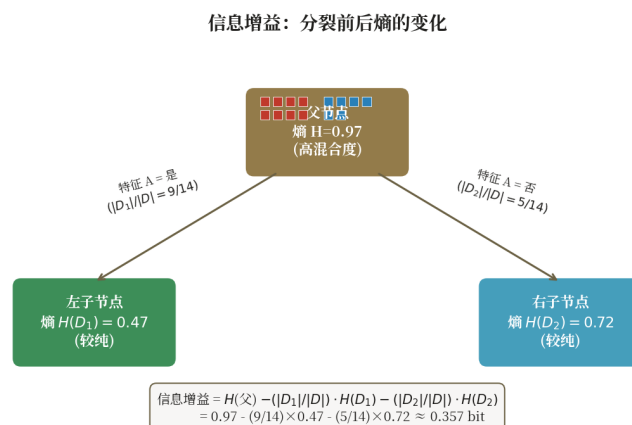


图 3.1 信息增益示意：分裂前父节点熵高，分裂后子节点熵低，差值即为信息增益

### 3.4 由度量准则推导出算法

有了"分裂使熵下降"这一优化目标后，决策树算法几乎是自动得出的——无需"设计"，只需顺着该目标递归推进即可。

具体流程如下：从根节点出发，对当前节点内所有样本计算每个特征的信息增益，选择信息增益最大的特征进行分裂；分裂得到若干子节点后，对每个子节点重复同样操作——这构成一个**递归**。递归终止条件有三：其一，子节点已纯（熵为零，所有样本同类）；其二，已无可用特征；其三，子节点样本数过少（少于阈值），不再值得分裂。终止后，该子节点成为**叶节点**，以其内多数样本的类别作为预测标签。

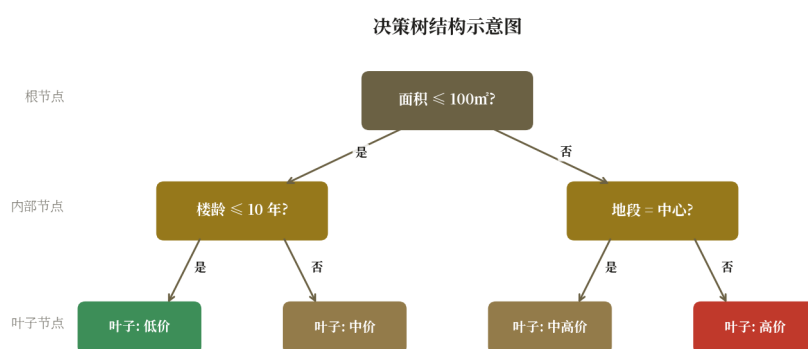


图 3.2 决策树结构示意图：根节点 → 内部节点 → 叶子节点，每条路径对应一条规则

上述流程的本质是分层条件划分——每一层选择一个最优特征进行分裂，逐步将样本空间划分为若干纯度递增的子区域。从根到任意叶子的路径即一条可读规则，如"年龄  $< 30$  且 收入  $> 1$  万 → 批准"。这是决策树在合规审计要求严格的行业广受采用的根本原因。

### 3.5 递归分裂的代价：过拟合风险

上述递归流程在理论上完美，但实际运行将出现问题：若不限制终止条件，树将持续分裂至每个叶节点都纯净为止。其后果是模型对训练样本过度拟合——即使“居住于 A 街道且年龄 31 岁”这类仅出现一次的边角样本，也会被分配一条专属分支。结果是训练集准确率达到 100%，但在新数据上性能急剧下降。这正是过拟合在决策树上的典型表现。

抑制过拟合的策略是限制树的生长规模，分为两类。其一为**预剪枝**（pre-pruning）——在树生长过程中提前终止。常用停止条件包括：限制最大深度（max\_depth）、限制节点最少样本数（min\_samples\_split）、限制叶节点最少样本数（min\_samples\_leaf）、限制最大叶节点数（max\_leaf\_nodes）。预剪枝实现简单、效率高，但存在过早停止导致欠拟合的风险。

其二为**后剪枝**（post-pruning）——先让树充分生长，再自底向上回溯：若将某内部节点替换为叶节点（即移除其下子树）后验证集性能反而提升，则执行剪枝。代表方法是 CART 的 Cost-Complexity Pruning（对应 sklearn 的 ccp\_alpha 参数）。后剪枝通常精度更高，但计算开销较大。

**[技巧]** sklearn 同时支持两种剪枝：通过 max\_depth 系列参数实现预剪枝，通过 ccp\_alpha 实现后剪枝。实践中常先以 max\_depth 控制树规模，再以 GridSearchCV 调优 ccp\_alpha 进行精修。

### 3.6 三大经典算法：同一框架下的三种变体

前文以“信息增益”为准则描述了 ID3 的核心。然而 ID3 存在一个明显缺陷：**偏向取值较多的特征**。极端情形——若以“身份证号”作为特征，分裂后每个子节点仅含 1 个样本，全部纯净，信息增益达到最大值；但该特征对预测毫无意义。

C4.5 正是为修正此缺陷而提出的变体。其思路是将信息增益除以“分裂信息”（SplitInfo）——该量度量“特征本身将数据切分的细碎程度”，对取值多的特征施加惩罚。由此得到增益率（Gain Ratio）：

$$\text{GainRatio}(D, A) = g(D, A) / \text{SplitInfo}(A), \text{SplitInfo}(A) = - \sum_v (|D_v|/|D|) \cdot \log_2(|D_v|/|D|)$$

C4.5 同时支持连续特征（通过二分离散化）、缺失值处理与剪枝，是 ID3 的全面升级版。

CART（Classification and Regression Tree）采用另一变体路径。其不使用熵而采用**基尼系数**，避免对数运算以提升效率：

$$\text{Gini}(D) = 1 - \sum_k p_k^2$$

基尼系数的数学含义是：从 D 中随机抽取两个样本，它们属于不同类别的概率。全为同类时 Gini = 0；各类均匀分布时 Gini 接近最大值。其表达的不确定性程度与熵一致，但计算开销更低。此外，CART 与 ID3/C4.5 的一大差异在于：CART **只生成二叉树**（每个节点仅做二分裂），且既可用于分类（基尼系数）也可用于回归（用平方误差作为分裂准则）。sklearn 的 DecisionTreeClassifier 即 CART 的实现。

## 3.7 算法实现：递归建树

将上述流程实现为代码，决策树的核心即一个**递归函数**：在每个节点选择最优分裂特征与阈值，将数据分为两份，对每份再递归调用自身。

### 3.7.1 数据结构

- **TreeNode**: { feature (分裂特征索引), threshold (分裂阈值), left (左子树), right (右子树), label (叶节点类别) }
- **参数**: max\_depth (最大深度), min\_samples\_split (最小分裂样本数)

### 3.7.2 fit(X, y) — 递归建树

**输入**: 特征矩阵 X, 标签 y, 当前深度 depth

**递归过程**:

- 1. 若满足停止条件 (纯度足够/深度达限/样本太少) → 创建叶节点, label = 众数(y)
- 2. 否则遍历每个特征 j 和每个候选阈值 t:
  - (a) 按阈值分割: 左集 =  $\{(x,y) \mid x_j \leq t\}$ , 右集 = 其余
  - (b) 算分裂后基尼系数:  $G = (|左|/n) \cdot \text{Gini}(左) + (|右|/n) \cdot \text{Gini}(右)$
  - (c) 选使 G 最小的  $(j^*, t^*)$
- 3. 创建内部节点, 存  $(j^*, t^*)$ , 递归调用 fit(左集, depth+1) 与 fit(右集, depth+1)

### 3.7.3 predict(X) — 树遍历

对每个样本 x: 从根节点出发, 按  $x_{\text{feature}} \leq \text{threshold}$  走左/右子树, 到达叶节点返回 label。

### 3.7.4 复杂度

- **训练**:  $O(n \cdot d \cdot \log n)$  每层, 共  $O(n \cdot d \cdot \log^2 n)$
- **预测**:  $O(\log n)$  单样本, 树深度为  $O(\log n)$

**[提示]** scikit-learn 的 CART 实现采用排序+前缀和加速分裂搜索, 比朴素遍历快数倍。

## 3.8 手算：用信息增益选分裂特征

以一个最小例子演示。下表是 6 位贷款申请人的简化数据, 类别 Y=批准 / N=拒绝, 问应优先按哪个特征分裂?

| ID | 有工作 | 信用好 | 类别 |
|----|-----|-----|----|
| 1  | 否   | 否   | N  |
| 2  | 否   | 是   | N  |
| 3  | 是   | 是   | Y  |
| 4  | 是   | 否   | Y  |

| ID | 有工作 | 信用好 | 类别 |
|----|-----|-----|----|
| 5  | 否   | 是   | N  |
| 6  | 是   | 是   | Y  |

解：根节点 D 含 6 个样本，3 Y 3 N，熵  $H(D) = 1 \text{ bit}$ 。

按"有工作"分裂：是 $\rightarrow\{3,4,6\}$  全 Y，熵 0；否 $\rightarrow\{1,2,5\}$  全 N，熵 0。条件熵  $H(D|\text{有工作}) = (3/6) \times 0 + (3/6) \times 0 = 0$ ，信息增益  $= 1 - 0 = 1 \text{ bit}$ 。

按"信用好"分裂：是 $\rightarrow\{2,3,5,6\} = \{N,Y,N,Y\}$ ，熵 1 bit；否 $\rightarrow\{1,4\} = \{N,Y\}$ ，熵 1 bit。条件熵  $H(D|\text{信用好}) = (4/6) \times 1 + (2/6) \times 1 = 1 \text{ bit}$ ，信息增益  $= 1 - 1 = 0 \text{ bit}$ 。

结论："有工作"信息增益 1 bit 远大于"信用好"的 0 bit，应优先按"有工作"分裂。分裂后两个子节点已纯，递归终止，得到仅含根节点的简单决策树。

该示例清晰展示了"分裂后纯度"的量化判据——"有工作"一刀将两类分开，故信息增益最大；"信用好"分裂后两侧仍混合，等于未提供信息，故增益为零。决策树即依据这套度量准则自动选择最优分裂特征。

### 3.9 应用实例：贷款审批决策树

某银行历史贷款数据集含 1000 条记录，特征包括年龄、收入、职业、负债率、征信评分、贷款金额、期限等 12 个字段，目标为预测是否违约。采用 sklearn DecisionTreeClassifier 训练，设置 max\_depth=5 以保证可解释性，得到的决策树典型规则如下：

- "负债率  $> 35\%$  且 征信评分  $< 680 \rightarrow$  拒绝"（高风险路径）
- "负债率  $\leq 35\%$  且 月收入  $> 8000 \rightarrow$  批准"（低风险路径）
- "负债率  $\leq 35\%$  且 月收入  $\leq 8000$  且 工龄  $< 2$  年  $\rightarrow$  拒绝"

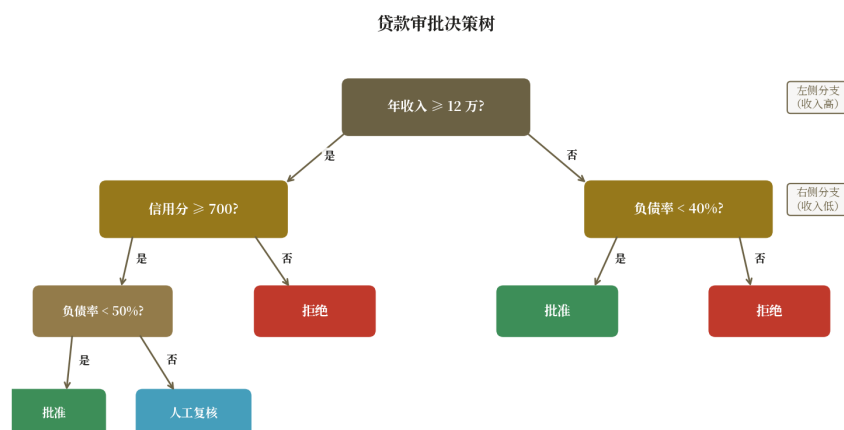


图 3.3 贷款审批决策树：根节点为负债率，深色叶节点代表批准、浅色代表拒绝

该树在测试集上准确率约 82%，AUC 0.78。对于"必须可向审计员解释"的金融场景，该性能已足够。若需进一步提升，可将多棵决策树组合为集成模型——随机森林（Bagging 思路）



或 GBDT/XGBoost/LightGBM（Boosting 思路）——通常可将 AUC 推至 0.85 以上，代价是牺牲部分可解释性。但无论集成多复杂，其底层基本构件仍是本章所述的决策树。

## 3.10 代码实现：sklearn DecisionTreeClassifier

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# -- 加载鸢尾花数据集 --
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# -- 训练决策树 (CART, 基尼系数, max_depth=3) --
clf = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_leaf=5,
    random_state=42,
)
clf.fit(X_tr, y_tr)

# -- 评估 --
y_pred = clf.predict(X_te)
print(f"测试集准确率: {accuracy_score(y_te, y_pred):.3f}")
print(classification_report(y_te, y_pred,
    target_names=load_iris().target_names))

# -- 可视化决策树 --
plt.figure(figsize=(12, 8))
plot_tree(clf, feature_names=load_iris().feature_names,
    class_names=load_iris().target_names,
    filled=True, rounded=True)
plt.title("Iris Decision Tree (max_depth=3)")
plt.tight_layout()
plt.savefig("iris_tree.png", dpi=120)

# -- 查看特征重要性 --
for name, imp in zip(load_iris().feature_names,
    clf.feature_importances_):
    print(f"{name:20s} importance={imp:.3f}")
```

## 3.11 现代发展：从单棵树到森林

单棵决策树可解释性强，但精度受限且易过拟合。现代工业界的核心发展方向是**集成学习**——以多棵树组合弥补单树的不足。

- **随机森林 (Random Forest)**：Bagging 思想，训练多棵独立决策树并投票/平均输出。降低方差、抗过拟合，开箱即用。
- **GBDT**：Boosting 思想，每棵新树拟合前一棵的残差（负梯度）。代表实现：sklearn GradientBoostingClassifier、XGBoost、LightGBM、CatBoost。

- **XGBoost / LightGBM / CatBoost**: 第三代梯度提升框架, 引入二阶导数、直方图近似、Leaf-wise 生长、GPU 加速等优化, 长期统治 Kaggle 表格数据竞赛。
- **可解释 Boosting (EBM)**: 保留决策树可解释性, 精度接近 GBDT, 是"既要又要"的折中方案。
- **软决策树**: 用神经网络模拟决策树, 支持端到端反向传播训练, 是树模型与深度学习融合的方向。

上述方法形式各异, 但其底层基本构件仍是本章所述的"选信息增益最大特征递归分裂"的决策树。掌握单棵树的原理, 是理解所有现代集成方法的前提。

## 第四章 聚类分析

### 4.1 聚类问题概述

聚类 (Clustering) 是无监督学习的核心任务。其目标是在没有标签的条件下, 将样本集合划分为若干簇, 使得**组内样本相似度高、组间样本相似度低**。与监督学习不同, 聚类没有预先给定的"正确答案", 算法需自行从数据中发现自然分组结构。

以电商客户分群为例: 营销部门希望将上百万用户划分为"价格敏感型""品牌忠诚型""冲动消费型"等若干群体, 以支撑精准营销决策。然而这些用户**没有标签**——运营方事先并不知道每位用户的类型, 仅有其点击、购买、停留时长等行为日志。此类问题正是聚类的典型应用场景。

聚类方法的应用范围广泛: 客户分群 (基于 RFM 特征的用户细分)、异常检测 (与任何簇都不相似的孤立样本)、图像分割 (按颜色或纹理将像素分组)、文档主题发现 (按词频共现将文档聚类)、生物信息学 (基因表达模式分组) 等。

聚类方法的设计围绕两个根本问题展开: 其一, **何为相似**——是欧氏距离、曼哈顿距离, 还是余弦相似度? 其二, **如何由相似度形成簇**——是按中心代表、按密度连通, 还是按层级合并? 不同的回答对应不同的聚类族:

- **划分式聚类**: 将样本直接划分为 K 个互斥簇, 以簇中心代表该簇。代表算法: K-Means。
- **层次式聚类**: 通过逐层合并或分裂构建聚类树 (dendrogram), 用户按需剪枝得到最终簇数。代表算法: Agglomerative Clustering。
- **密度式聚类**: 将高密度连通区域定义为簇, 可识别任意形状的簇并标记噪声。代表算法: DBSCAN。

本章依次介绍这三类方法。三者并非互斥或递进关系, 而是对"相似度"与"成簇准则"的不同数学选择。之后给出统一的内部评估指标, 并以客户分群、颜色量化、异常检测三类应用收尾。

### 4.2 K-Means: 基于中心的划分式聚类

K-Means 是最经典的划分式聚类算法。其核心假设是: 每个簇可由一个中心点 (centroid) 代表, 样本归属于距其最近的中心点所在簇。本节从该假设出发, 经直觉推导、迭代步骤、K 选择到局限性, 依次展开。

#### 4.2.1 直觉推导: 从"相似度归堆"到"中心代表簇"

聚类的最朴素目标是"将相似样本归为一堆"。相似度最直接的度量为**距离**——两个样本的特征向量越接近，二者越相似。在欧氏空间中常用欧氏距离度量。

由此"将相似的放一起"可翻译为：使一个簇内所有样本到该簇的"代表点"距离尽量小。该代表点即簇的中心。若事先确定簇数为  $K$ ，且已知每个簇的中心  $\mu_1, \mu_2, \dots, \mu_K$ ，则一个样本应归属于距其最近的中心所在的簇——这是 K-Means 的核心分配准则。

然而初始时刻簇中心是未知的。处理思路是**迭代估计**：先给定  $K$  个初始中心，根据当前中心完成样本分配；再根据分配结果重新计算中心；反复迭代直至收敛。由此即可推导出 K-Means 的完整算法流程。

#### 4.2.2 算法步骤：分配-更新的迭代过程

基于上述推导，K-Means 的算法步骤如下：

- **步骤 1（初始化）**：在数据空间中放置  $K$  个中心点  $\mu_1, \dots, \mu_K$ 。常见做法是从样本中随机选取  $K$  个，或采用 K-Means++ 智能初始化。
- **步骤 2（分配步）**：对每个样本  $x_i$ ，计算其到各中心的距离，将其分配给最近的中心所在的簇： $C_k = \{x_i \mid k = \operatorname{argmin}_j \|x_i - \mu_j\|^2\}$
- **步骤 3（更新步）**：对每个簇，重新计算其所有样本的均值作为新中心： $\mu_k \leftarrow (1/|C_k|) \cdot \sum_{x \in C_k} x$
- **步骤 4（收敛判定）**：若中心不再变化（或变化小于阈值），则收敛；否则返回步骤 2。

该迭代过程在数学上是最小化**簇内平方误差和**（Within-Cluster Sum of Squares, WCSS）：

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

其中  $C_k$  是第  $k$  个簇的样本集合， $\mu_k$  是其中心。J 即"所有样本到自身簇中心的距离平方和"，越小则聚类越紧凑。需注意：该优化问题是 NP-hard，迭代算法仅保证收敛到**局部最优**，不一定是全局最优。因此实践中常采用"多次随机初始化取最优结果"的策略（sklearn 的 `n_init` 参数即实现此机制）。

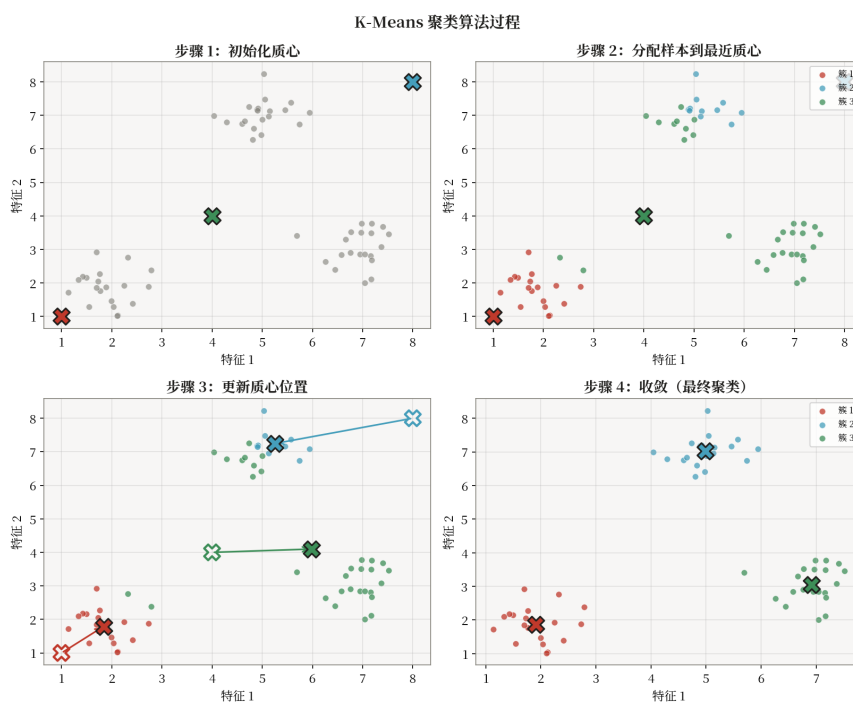


图 4.1 K-Means 迭代过程：初始化 → 分配 → 更新 → 收敛

**[提示]** sklearn 默认采用 **K-Means++** 智能初始化——第一个中心随机选取，后续每个中心按“距离平方”的概率分布选取，使初始中心彼此尽量分散。该策略能显著加速收敛、降低陷入较差局部最优的概率。

### 4.2.3 K 的选择：手肘法与轮廓系数

K-Means 需事先指定簇数  $K$ ，而真实数据的自然簇数事先未知。由此需采用数据驱动的方法估计  $K$ 。常用方法有二。

其一为**手肘法**（Elbow Method）。对  $K = 1, 2, \dots, K_{\max}$  分别运行 K-Means，计算各自的簇内平方误差和 SSE。 $K$  越大则 SSE 越小（极端  $K=n$  时  $SSE=0$ ，每样本自成一类）；然而 SSE- $K$  曲线常出现明显“肘部”——肘部之前 SSE 下降迅速，之后下降趋缓。该肘部对应的  $K$  即“再增加一类收益已不显著”的临界点。

其二为**轮廓系数**（Silhouette Coefficient）。对每个样本计算两个量： $a$  为该样本到本簇其他样本的平均距离（越小，说明本簇越紧凑）； $b$  为该样本到最近其他簇内样本的平均距离（越大，说明与其他簇越分离）。轮廓系数定义为  $s = (b - a) / \max(a, b)$ ，取值于  $[-1, 1]$ ：接近 1 表示该样本分类合理，接近 0 表示处于簇边界，接近 -1 表示分类错误。取所有样本  $s$  的平均作为整体聚类质量度量，使平均轮廓系数最大的  $K$  即为最佳选择。

手肘法直观但肘部有时不明显，轮廓系数严格但计算开销较大。实践中常将二者结合——先用手肘法粗筛  $K$  的范围，再用轮廓系数做精修。

### 4.2.4 局限性：凸簇假设与异常敏感性

K-Means 形式简洁，但存在若干数学假设带来的固有局限。

- **凸簇假设**：K-Means 隐含假设每个簇为**凸集**（如球形、超球体），且各簇大小相近。其原因是分配准则基于“到中心的距离”，每个簇的势力范围即以中心为圆心、半径相等的区域。因此对于细长条形、月牙形、环形等非凸形状的簇，K-Means 难以正确识别。
- **需预先指定 K**：K 由用户事先给定，无法从数据自动推断。
- **异常点敏感**：均值更新对异常点高度敏感——少量远离簇心的异常点会显著拖动中心位置。
- **对初始化敏感**：不同初始中心可能导致不同的局部最优解。

上述局限源于“用中心代表簇”这一基本建模选择。若数据本身不符合凸簇假设，可改用基于距离的层级式聚类（4.3 节）或基于密度的聚类（4.5 节）——它们采用完全不同的相似度建模方式，是独立而非递进的聚类思路。

## 4.3 层次聚类：基于距离的层级式聚类

层次聚类（Hierarchical Clustering）采用与 K-Means 不同的建模思路——不直接给出 K 个簇的划分，而是将聚类过程**完整记录**，构建一棵聚类树（dendrogram）。用户在树的不同高度剪枝，即可得到不同粒度的簇划分。

层次聚类按方向分两种。**自底向上（聚合型 Agglomerative）**最常用：初始时每个样本自成一类，每步将“最相似”的两类合并，直至全部合并为一类。**自顶向下（分裂型 Divisive）**则相反：初始所有样本为一类，每步将一类分裂为两子类，直至每类仅含一个样本。分裂型计算开销较大，实际中较少使用。

聚合型层次聚类的关键在于**如何度量两个簇之间的距离**，称为 linkage 函数。常用的四种 linkage 及其特性如下：

- **single linkage**（最近邻距离）：取两簇间样本的最小距离。易产生“链式效应”，使相邻簇被逐步吞并。
- **complete linkage**（最远邻距离）：取两簇间样本的最大距离。倾向于生成紧凑球形簇。
- **average linkage**（平均距离）：取两簇间所有样本对的平均距离。是上述两者的折中。
- **ward linkage**（最小平方误差增量）：选择使合并后方差增量最小的两簇合并。在多数数据集上效果最优。

dendrogram 构建完成后，可通过设定距离阈值或目标簇数进行剪枝，得到最终聚类结果。层次聚类的优势在于：无需事先指定 K，且可在不同粒度下观察数据结构。其代价是计算复杂度较高——朴素实现为  $O(n^3)$ ，使用优先队列可降至  $O(n^2 \log n)$ 。

## 4.4 评估指标：无标签条件下的聚类质量度量

聚类面临一个方法论难题：没有真实标签，无法像分类那样直接计算准确率。因此只能采用**内部指标**——基于簇内紧密度与簇间分离度度量聚类质量。常用的内部指标有以下几个：

- **轮廓系数（Silhouette）**： $s = (b-a)/\max(a,b)$ ，a 为同簇平均距离，b 为到最近其他簇的平均距离。取值  $[-1, 1]$ ，越大越好； $>0.5$  表示聚类结构合理。4.2.3 节选 K 用的即是此指标。



- **DB 指数 (Davies-Bouldin)**：度量簇内紧密度与簇间分离度的比值，越小越好。
- **CH 指数 (Calinski-Harabasz)**：簇间离散度与簇内离散度之比，越大越好。
- **SSE / 拐点法**：直接观察簇内方差和随 K 变化的曲线，寻找肘部位置。

**[提示]** 若进行对比实验且恰好有真实标签可用，则可采用**外部指标**：ARI (Adjusted Rand Index)、NMI (归一化互信息)、FMI (Fowlkes-Mallows Index)，取值均在  $[0, 1]$  之间，越大表示聚类结果与真实标签越一致。

## 4.5 DBSCAN：基于密度的聚类

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是与 K-Means、层次聚类并列的另一类聚类方法。其建模思路截然不同：不以"中心点"代表簇，而以"局部密度连通性"定义簇——高密度连通区域即为一簇，密度过低的孤立点视为噪声。该思路使得 DBSCAN 可识别任意形状的簇，并天然支持异常检测。

### 4.5.1 核心思想：基于局部密度而非中心距离

K-Means 通过"到中心点的距离"定义簇归属，这隐含假设每个簇为凸形且大小相近。DBSCAN 则完全放弃"中心"概念，转而采用**局部密度**作为成簇依据：若某区域内样本分布密集，则将该区域归为一个簇；若某样本周围密度过低、与任何密集区域均不连通，则视为噪声。

由此，DBSCAN 的簇形状由数据分布密度决定，而非预设的几何形状——月牙形、环形、不规则形状均可识别。同时，由于噪声点被显式标记，DBSCAN 天然具备异常检测能力。

### 4.5.2 两个参数： $\epsilon$ 与 MinPts

DBSCAN 仅依赖两个参数来定义"密集"：

- **$\epsilon$  (邻域半径)**：以每个点为圆心、 $\epsilon$  为半径的圆内所有点视为该点的邻居。 $\epsilon$  控制"邻域"的空间尺度。
- **MinPts (最小邻居数)**：一个点的  $\epsilon$  邻域内至少需含 MinPts 个点，方认为该点处于"密集区域"。MinPts 控制"密集"的密度阈值。

这两个参数共同确定"何为密集"—— $\epsilon$  定义空间范围，MinPts 定义数量阈值。二者组合即给出每个点的**密度**：邻域内邻居数越多，密度越高。实际中 MinPts 通常取  $d+1$  或  $2d-1$  ( $d$  为特征维度)， $\epsilon$  通过 k-距离图选取。

### 4.5.3 三类点：核心点、边界点、噪声点

给定  $\epsilon$  与 MinPts，每个点依据其邻域密度被划分为三类之一。判定规则如下：



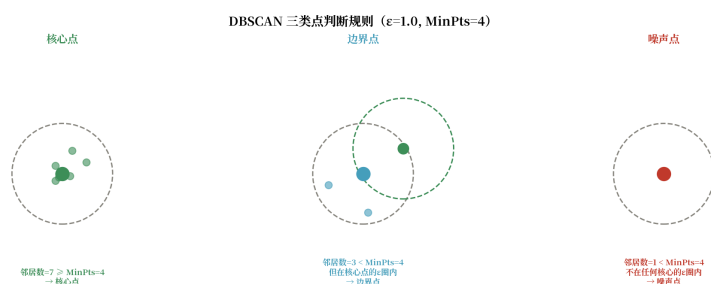


图 4.2 DBSCAN 三类点判断规则：左=核心点（圈内邻居数 $\geq \text{MinPts}$ ），中=边界点（邻居不够但落在核心点圈内），右=噪声点（孤立无伴）

**核心点 (Core Point)：**以该点为圆心的  $\epsilon$  邻域内点数  $\geq \text{MinPts}$ 。该点处于密集区域内部，是簇的"骨架"。

**边界点 (Border Point)：**自身  $\epsilon$  邻域内点数  $< \text{MinPts}$ （非核心），但其位于某个核心点的  $\epsilon$  邻域内。该点处于簇的边缘，被核心点"覆盖"。

**噪声点 (Noise Point)：**既非核心点，也不在任何核心点的  $\epsilon$  邻域内。该点远离任何密集区域，孤立无伴。

三类点的区分使得 DBSCAN 的输出包含两类信息：其一，密集区域内的样本被组织为若干簇；其二，稀疏区域的样本被显式标记为噪声。这是 DBSCAN 区别于 K-Means 的核心特征之一。

#### 4.5.4 算法流程：从核心点"密度可达"扩散成簇

基于三类点的定义，DBSCAN 的算法逻辑如下——核心机制是"从核心点出发，沿密度可达关系递归扩散"：

- **步骤 1：**遍历所有点，统计每个点的  $\epsilon$  邻居数，标记核心点（邻居数  $\geq \text{MinPts}$ ）。
- **步骤 2：**从未访问的核心点中任选一个 A，创建新簇，将 A 加入。
- **步骤 3：**考察 A 的所有  $\epsilon$  邻居：
  - → 若邻居 B 是核心点：将 B 的所有  $\epsilon$  邻居也加入待考察队列（此即"密度可达"的递归扩散）。
  - → 若邻居 B 不是核心点：B 为边界点，加入当前簇，但不再扩散。
- **步骤 4：**重复步骤 3，直至待考察队列为空——该簇"扩散完毕"。
- **步骤 5：**返回步骤 2，从未访问的核心点中选取下一个，开启新簇。
- **步骤 6：**所有点访问完毕后，未被任何簇包含的点即为噪声。

上述"密度可达"关系是 DBSCAN 的数学核心：若点 B 落在核心点 A 的  $\epsilon$  邻域内，则称 B 从 A "直接密度可达"；若存在点链  $P_1=A, P_2, \dots, P_n=B$ ，其中每个  $P_{i+1}$  从  $P_i$  直接密度可达，则称 B 从 A "密度可达"。一个簇即由所有从一个核心点密度可达的点组成。

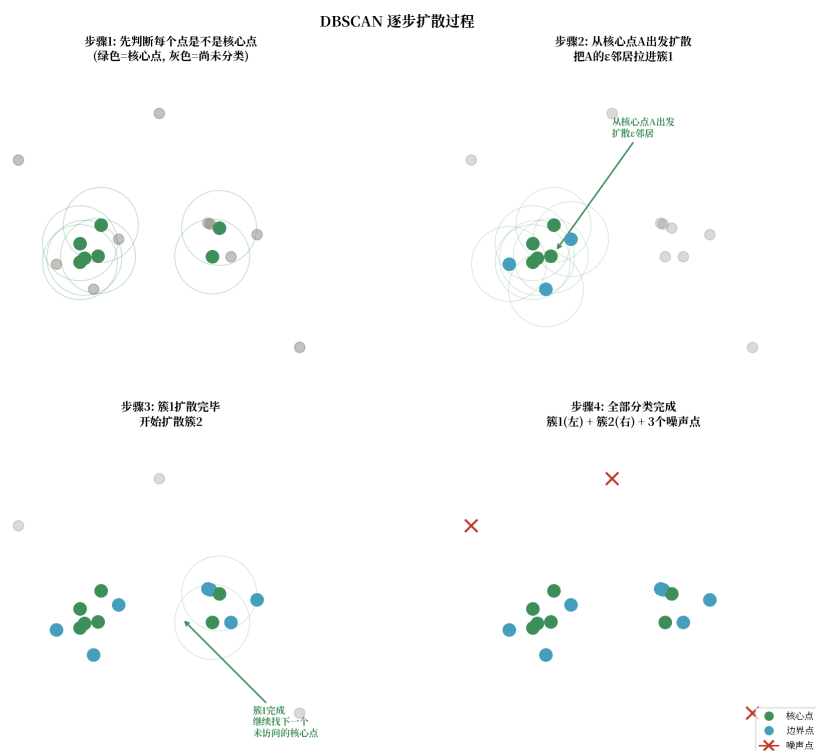


图 4.3 DBSCAN 逐步扩散过程: ①判断核心点 → ②从 A 扩散 → ③簇 1 完成开启新簇 → ④全部分类完毕

### 4.5.5 逐步计算实例

设  $\epsilon=0.8$ ,  $\text{MinPts}=4$ , 给定 8 个二维样本点。各点的坐标、 $\epsilon$  邻居数及类型如下表:

| 点 | x    | y    | $\epsilon$ 邻居数  | 类型  |
|---|------|------|-----------------|-----|
| A | -1.8 | 0.1  | 6 (B,C,D,E,F,G) | 核心点 |
| B | -1.5 | -0.3 | 5 (A,C,D,E,F)   | 核心点 |
| C | -1.2 | 0.4  | 4 (A,B,D,E)     | 核心点 |
| D | -1.6 | 0.5  | 5 (A,B,C,E,G)   | 核心点 |
| E | -1.4 | 0.0  | 5 (A,B,C,D)     | 核心点 |
| F | -2.0 | -0.2 | 3 (A,B,E)       | 边界点 |
| G | -1.0 | 0.8  | 2 (A,D)         | 边界点 |
| H | 3.0  | -2.0 | 0               | 噪声点 |

表 4.1 DBSCAN 8 点数据 ( $\epsilon=0.8$ ,  $\text{MinPts}=4$ )

**执行过程:**

**第 1 步:** 遍历所有点, 计算每个点的  $\epsilon$  邻居数。A 有 6 个邻居  $\geq 4 \rightarrow$  核心点; B 有 5  $\rightarrow$  核心; C 有 4  $\rightarrow$  核心; D 有 5  $\rightarrow$  核心; E 有 5  $\rightarrow$  核心。F 有  $3 < 4 \rightarrow$  非核心; G 有  $2 < 4 \rightarrow$  非核心; H 有 0  $\rightarrow$  非核心。

**第 2 步:** 选未访问的核心点 A, 创建簇 1, 将 A 加入。考察 A 的邻居 {B, C, D, E, F, G}。

**第3步：** B是核心点 → 将B的邻居 {A, C, D, E, F} 也加入待考察队列。C是核心点 → 将C的邻居 {A, B, D, E} 加入队列。D是核心点 → 将D的邻居 {A, B, C, E, G} 加入队列。E是核心点 → 将E的邻居 {A, B, C, D} 加入队列。F不是核心点 → F为边界点，加入簇1，不再扩散。G不是核心点 → G为边界点，加入簇1，不再扩散。

**第4步：** 待考察队列已空，簇1 = {A, B, C, D, E, F, G}，扩散完毕。

**第5步：** 寻找下一个未访问的点 → H。H非核心点且不在任何核心点的  $\varepsilon$  邻域内 → 标记为噪声。

**结果：** 簇1 = {A, B, C, D, E, F, G}（其中A-E为核心点，F-G为边界点），H为噪声点。

### 4.5.6 DBSCAN 的优势与局限

DBSCAN 的优势源于其密度建模思路：

- **无需预先指定簇数 K：** 簇数由数据自身密度结构决定，密集区域数即簇数。
- **可识别任意形状的簇：** 月牙形、环形、不规则形状均可识别，因为成簇依据是密度连通性而非几何中心。
- **天然支持异常检测：** 噪声点即异常候选，无需额外算法或阈值设定。

其局限同样源于密度建模的假设：

- **对  $\varepsilon$  与 MinPts 敏感：** 参数选择不当将导致过度合并（全部样本归一簇）或过度分裂（全部样本为噪声）。
- **密度差异大的数据表现差：** 若数据中存在多个密度差异显著的密集区域，一组 ( $\varepsilon$ , MinPts) 难以同时适配。
- **高维效果差：** 高维空间中距离的区分度下降（维度灾难）， $\varepsilon$  邻域难以有效定义。

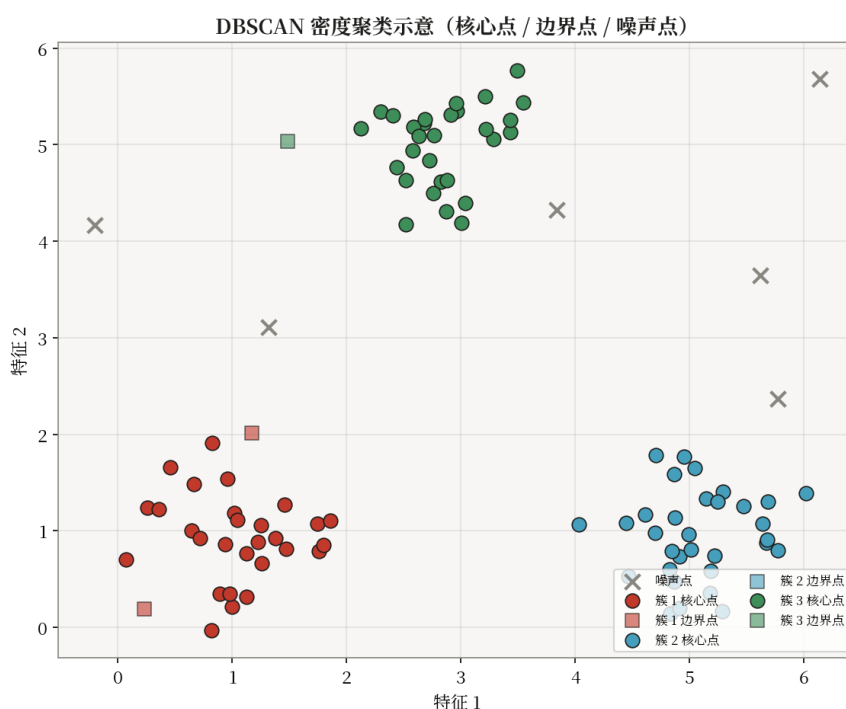


图 4.4 DBSCAN 聚类效果：核心点稠密成簇，边界点环绕，噪声点散落

## 4.6 算法实现：K-Means 伪代码

将 4.2 节的推导收拢为代码实现，K-Means 的核心即"初始化 → 分配 → 更新 → 收敛"的迭代循环。对外暴露三个方法：fit（训练，求中心）、predict（给新样本分配簇）、transform（计算到各中心的距离）。

### 4.6.1 数据结构

- **cluster\_centers\_**: 矩阵  $[k, d]$ ,  $k$  个簇中心的坐标
- **labels\_**: 向量  $[n]$ , 每个样本的簇分配
- **inertia\_**: 标量, 簇内方差和（收敛指标）

### 4.6.2 fit(X) — Lloyd 迭代

输入: 数据  $X \in \mathbb{R}^{n \times d}$ , 簇数  $k$ , 最大迭代  $T$

过程:

- 1. **K-Means++ 初始化**: 随机选第 1 个中心; 对  $i = 2, \dots, k$ : 算各点到已选中心的最小距离  $D(x)$ , 按概率  $P(x) \propto D(x)^2$  选下一个中心
- 2. **迭代** ( $t = 1, \dots, T$ ):
  - (a) **分配步**: 对每个  $x_i$ ,  $\text{labels}_i \leftarrow \operatorname{argmin}_j \|x_i - c_j\|^2$
  - (b) **更新步**: 对每个簇  $j$ ,  $c_j \leftarrow \operatorname{mean}(\{x_i \mid \text{labels}_i = j\})$
  - (c) 若中心不再变化则收敛, 提前终止
- 3. 计算  $\text{inertia}_t = \sum_i \|x_i - c_{\text{labels}_i}\|^2$

### 4.6.3 predict(X) 与 transform(X)

$$\text{label}(x) = \operatorname{argmin}_j \|x - c_j\|^2$$

transform 返回  $[n, k]$  距离矩阵  $D_{ij} = \|x_i - c_j\|$ 。

### 4.6.4 复杂度

- **训练**:  $O(nkdT)$ ,  $T$  为迭代次数
- **预测**:  $O(nkd)$ , 计算到  $k$  个中心的距离

**[提示]** scikit-learn 采用 Elkan 三角不等式加速, 避免部分距离计算, 比朴素实现快 2-10 倍。

## 4.7 计算示例：5 个一维点的 K-Means

以一个最小例子演示 K-Means 的完整迭代过程。设 5 个一维样本  $X = \{1, 2, 8, 9, 25\}$ ,  $K=2$ , 初始中心  $c_1=1, c_2=2$ 。

**第 1 轮分配**（按到中心距离）：

$1 \rightarrow C_1$  ( $|1-1|=0$ );  $2 \rightarrow C_2$  ( $|2-2|=0$ );  $8 \rightarrow C_2$  (6 vs 7);  $9 \rightarrow C_2$  (7 vs 8);  $25 \rightarrow C_2$  (23 vs 24)

得  $C_1=\{1\}$ ,  $C_2=\{2,8,9,25\}$

**第 1 轮更新:**  $c_1=1$ ,  $c_2=\text{mean}(2,8,9,25)=44/4=11$

**第 2 轮分配** ( $c_1=1$ ,  $c_2=11$ ) :

$1 \rightarrow C_1$  (0 vs 10);  $2 \rightarrow C_1$  (1 vs 9);  $8 \rightarrow C_2$  (7 vs 3);  $9 \rightarrow C_2$  (8 vs 2);  $25 \rightarrow C_2$  (24 vs 14)

得  $C_1=\{1,2\}$ ,  $C_2=\{8,9,25\}$

**第 2 轮更新:**  $c_1=\text{mean}(1,2)=1.5$ ,  $c_2=\text{mean}(8,9,25)=42/3=14$

**第 3 轮分配** ( $c_1=1.5$ ,  $c_2=14$ ) :

$1 \rightarrow C_1$  (0.5 vs 12.5);  $2 \rightarrow C_1$  (0.5 vs 11.5);  $8 \rightarrow C_2$  (6.5 vs 6);  $9 \rightarrow C_2$  (7.5 vs 5);  $25 \rightarrow C_2$  (23.5 vs 11)

得  $C_1=\{1,2\}$ ,  $C_2=\{8,9,25\}$  (与上轮相同, **已收敛**)

最终聚类  $C_1=\{1,2\}$ ,  $C_2=\{8,9,25\}$ ,  $\text{SSE} = (1-1.5)^2 + (2-1.5)^2 + (8-14)^2 + (9-14)^2 + (25-14)^2 = 0.25 + 0.25 + 36 + 25 + 121 = 182.5$ 。

**[注意]** 注意: 若初始中心改为  $c_1=1$ ,  $c_2=25$ , 将收敛至  $C_1=\{1,2,8,9\}$ ,  $C_2=\{25\}$ , SSE 仅 50——此即说明 K-Means 对初始中心敏感, 需多次初始化取最优。K-Means++ 智能初始化正是为缓解该问题而设计。

## 4.8 应用实例: 客户分群、颜色量化与异常检测

聚类在工业界有大量成熟应用, 下面列举三个典型场景。

### 4.8.1 客户 RFM 分群

回到开头的电商客户分群问题。行业中最常用的特征框架是**RFM**: R (Recency, 最近一次购买距今天数)、F (Frequency, 购买次数)、M (Monetary, 累计消费金额)。三维特征合起来基本刻画了客户的"价值轮廓"。

某电商平台 10 万活跃用户, 对 (R, F, M) 三维特征先做对数变换与标准化, 用 K-Means 聚为 4 簇, 业务团队根据各簇均值打标签如下:

- **簇 1: 高价值忠诚客** (R 低、F 高、M 高, 约 15%) ——重点维系, 主推会员升级。
- **簇 2: 潜力唤醒客** (R 高、F 中、M 中, 约 25%) ——发优惠券刺激复购。
- **簇 3: 低频大额客** (R 高、F 低、M 高, 约 10%) ——主推新品, 提升频次。
- **簇 4: 边缘流失客** (R 高、F 低、M 低, 约 50%) ——低成本触达或放弃。

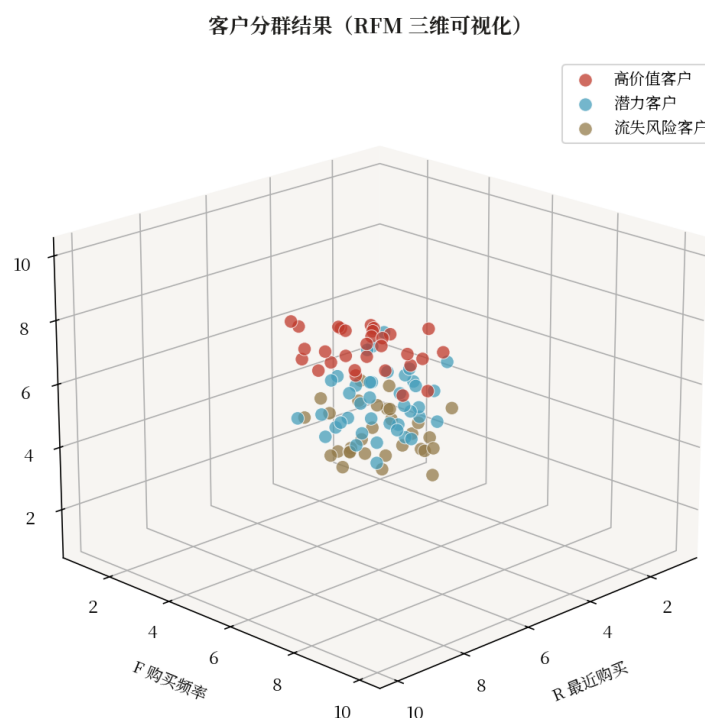


图 4.5 客户 RFM 分群结果：4 个簇在 R-F-M 三维空间中的分布

### 4.8.2 图像颜色量化压缩

K-Means 在图像压缩中有一个巧妙应用——颜色量化。一张照片可能含 16 万种颜色，但人眼可分辨的颜色数远低于此。若能将颜色数压缩到 16 种而视觉差异不大，文件体积可减少 80% 以上。

具体做法：将每个像素的 R-G-B 三维向量视为一个样本，所有像素构成大矩阵（ $H \times W$  行，3 列），用 K-Means 聚为 K 类——K 个簇中心即 K 种“代表色”，构成调色板。然后将原图每个像素的颜色替换为其所属簇的中心颜色，即完成“颜色量化”。sklearn 直接提供接口：`cluster.KMeans` 对像素矩阵聚类，`centers` 即调色板，`labels` 即索引图。

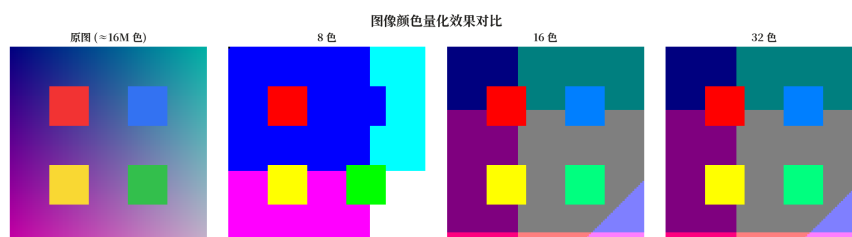


图 4.6 颜色量化效果对比：原图（24 位真彩色）与 K=16 量化后的压缩图

### 4.8.3 异常检测

聚类算法天然适合异常检测——因为“异常点”本质上即“与任何簇都不相似的孤立样本”。常用思路有三种：

- **K-Means 距离法**：计算每个样本到其簇中心的距离，距离大于阈值（如 95 分位数）的视为异常。



- **DBSCAN 噪声法**：直接利用 DBSCAN 标记的噪声点作为异常，无需调阈值——这是 4.5 节 DBSCAN 天然输出噪声的特性直接应用。

- **孤立森林 (Isolation Forest)**：专为异常检测设计的树模型，用随机划分路径长度判定异常，效率极高。

典型应用场景包括信用卡欺诈检测、网络入侵检测、工业设备故障预警、医疗指标异常监控等。这些场景中“异常样本”通常极少 (<1%)，监督学习难以收集足够的正样本标签，因此基于聚类的无监督方法反而成为更合适的选择。

## 4.9 代码实现：sklearn KMeans 与 DBSCAN

```
import numpy as np
from sklearn.cluster import KMeans, DBSCAN
from sklearn.datasets import make_moons, make_blobs
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# -- 生成数据：300 个 blob 样本 + 200 个月牙样本 --
X_blob, _ = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.6, random_state=42)
X_moon, _ = make_moons(n_samples=200, noise=0.08,
                        random_state=42)

# -- K-Means：必须先标准化 --
X_s = StandardScaler().fit_transform(X_blob)
km = KMeans(n_clusters=4, init="k-means++",
            n_init=10, random_state=42)
labels_km = km.fit_predict(X_s)
print(f"K-Means 轮廓系数: "
      f"{silhouette_score(X_s, labels_km):.3f}")

# -- DBSCAN：擅长非凸形状，无需指定 K --
X_m = StandardScaler().fit_transform(X_moon)
db = DBSCAN(eps=0.3, min_samples=5)
labels_db = db.fit_predict(X_m)
n_clusters = len(set(labels_db)) - (1 if -1 in labels_db else 0)
n_noise = list(labels_db).count(-1)
print(f"DBSCAN 发现 {n_clusters} 个簇，噪声点 {n_noise} 个")

# -- 用肘部法确定最佳 K --
sse = []
for n in range(1, 11):
    km_n = KMeans(n_clusters=n, n_init=10,
                  random_state=42).fit(X_s)
    sse.append(km_n.inertia_)
print("SSE 曲线:", [round(s, 1) for s in sse])
# 肘部位置对应最佳 K，结合轮廓系数验证
```

## 4.10 现代发展：聚类在深度学习时代

聚类虽是经典课题，但至今仍在快速演进。深度学习时代的几个值得关注方向如下：

- **深度聚类 (Deep Clustering)**：用深度网络（如 AutoEncoder）先学习表示再聚类，代表方法 DEC、DeepCluster、SCAN。在图像、文本等高维数据上效果远超传统 K-Means。

- **谱聚类 (Spectral Clustering)**：基于图拉普拉斯矩阵的特征向量做聚类，可识别任意形状的簇，常用于社交网络社区发现。
- **HDBSCAN**：DBSCAN 的层次扩展，自动选择密度阈值，对参数不敏感，已成为密度聚类的新标准。
- **主题模型**：LDA、BERTopic 本质上是文本数据上的软聚类，用于文档主题发现。
- **对比聚类**：结合对比学习与聚类的自监督方法（如 SwAV、MoCo-cluster），是表示学习与聚类融合的前沿方向。

上述方法虽名称各异，但内核均围绕本章讨论的两个根本问题——"何为相似"与"如何由相似形成簇"。K-Means 用中心定义相似，DBSCAN 用密度定义相似，深度聚类用学习到的表示定义相似——方法论一脉相承。掌握本章三类基本聚类思路，再去前沿方法便不会迷失方向。

## 第五章 集成学习

上一章我们讲了决策树。它的优点很直观：切几刀就把数据分干净了，还能告诉你是按哪几个特征切的，解释性极强。但决策树有一个让人头疼的毛病——它太“听话”了。训练数据里有什么，它就学什么，连噪声都一起学进去。于是稍微深一点的树，在训练集上能 100% 准确，换到测试集就掉到 70%——这就是经典的过拟合。

剪枝能缓解，但治标不治本。那我们换个思路：既然一棵树容易过拟合，能不能多建几棵不一样的树，让它们互相纠错？一个人的判断可能有偏差，但如果是 100 个人投票呢？——这就是**集成学习**（Ensemble Learning）的出发点：集成多个弱学习器可以获得比单个强学习器更好的泛化性能。

这个朴素想法之所以有用，背后其实有严格的数学支撑。统计学里把模型的误差拆成两部分：**偏差**（bias，模型假设错了多远）和**方差**（variance，换一份训练数据模型变多少）。决策树的问题是方差大——数据稍微一变，树形就跟着大变。而把多棵独立的树平均一下，方差会被显著地“削平”。这就是集成学习能奏效的根本原因。

### 5.1 三种把模型组合起来的思路

想法有了：要组合多个模型。但“怎么组合”才是关键。我们不妨退一步想——如果你要请 100 个人来帮忙判断一件事，这 100 个人该怎么组织？其实只有三种自然的方式。

**第一种：让他们各自独立判断，最后投票。**这对应**Bagging**（Bootstrap Aggregating）：让每个模型在不同的训练子集上独立训练，谁也不依赖谁，最后预测时投票或取平均。它的特点是**并行**——100 棵树可以同时训练，互不干扰，主要用来压制高方差模型（比如深决策树）的过拟合。

**第二种：让后面的人专攻前面的人做错的题。**这对应**Boosting**：模型一个接一个串行训练，后一个模型重点学前一个模型预测错的那些样本。每加一个模型，整体的错误就少一点。它是**串行的**，主要用来压低偏差——把一堆“勉强比随机猜好一点”的弱学习器叠起来，竟也能逼近强学习器。

**第三种：让一个“主席”学习怎么综合各位专家的意见。**这对应**Stacking**（Stacked Generalization）：让多个不同的基模型（比如树、SVM、神经网络）先各自预测，再训练一个“元学习器”专门学习“在什么情况下该信哪个模型”。它适合多源异构模型的组合，是 Kaggle 比赛最后冲分时的常见招数。

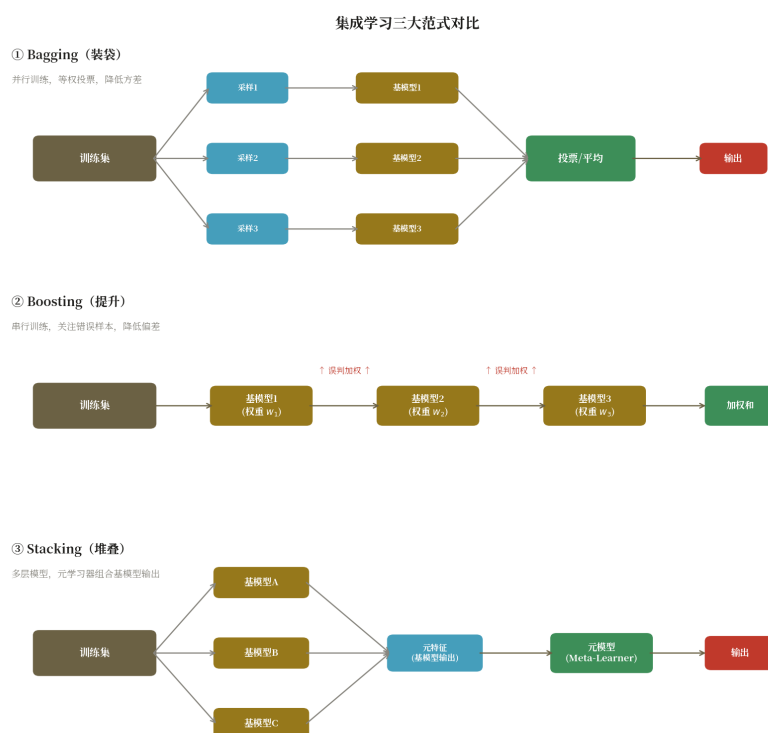


图 5.1 三大集成范式对比: Bagging 并行投票、Boosting 串行纠错、Stacking 分层组合

三种思路对应三种"团队协作方式", 也对应不同的适用场景。本章主线会落在 Bagging (随机森林) 和 Boosting (GBDT / XGBoost) 上, 因为它们是表格数据建模的两大主力; Stacking 我们在最后一节简单提一下。

## 5.2 Bagging 与随机森林: 从直觉一步步组装

先看 Bagging。它的核心问题就一个: **怎么让 100 棵树"足够不一样"**? 如果它们都长得一样, 投票也没用——100 个一模一样的人投票, 等于一个人投。所以 Bagging 的关键不在"投票"本身, 而在**怎么制造多样性**。

最自然的办法是**给每棵树喂不同的训练数据**。具体做法叫**自助采样 (bootstrap)**: 从  $N$  个样本里有放回地随机抽  $N$  次, 每次都从原来的  $N$  个里随机取一个, 抽完放回去再抽。这样得到的子集里, 有些样本会重复出现, 有些样本根本没被抽中。数学上可以证明, 每个子集大约包含原数据 63.2% 的不重复样本, 剩下的 36.8% 称为**袋外样本 (Out-Of-Bag, OOB)**——它们对这棵树来说是"没见过的数据", 正好可以拿来当免费的验证集。

有了 100 份不同的训练子集, 就能训出 100 棵不同的树。每棵树都不剪枝, 让它尽量深、尽量在子集上学到位。预测时让 100 棵树各自预测, 回归任务取平均、分类任务多数投票。数学上可以证明: 如果 100 棵树相互独立且方差都为  $\sigma^2$ , 平均后方差降到  $\sigma^2/100$ ——这就是 Bagging 削平方差的根源。

故事到这还没完。Bagging 只在数据上做了随机, 但有一个隐患: 如果数据里有几个特别"强"的特征 (比如风控里的"信用额度"), 那 100 棵树的根节点很可能都选这几个特征分裂, 结果 100 棵树其实长得很像, 投票效果就大打折扣。

**随机森林** (Random Forest, RF) 就是为解决这个隐患而生的。它在 Bagging 的基础上再加一层随机——**特征随机**：每棵树每次分裂一个节点时，不从全部  $d$  个特征里选最优，而是先随机抽  $m$  个特征（通常  $m = \sqrt{d}$ ），只在这  $m$  个里选。这样就算“信用额度”再强，也可能没被抽进某棵树的候选里，那棵树就不得不切到别的特征上去——树与树之间的相关性被进一步压低。

把三件事拼起来就是随机森林：**Bagging（数据随机）+ 决策树（高方差模型）+ 特征随机（树去相关）**。这个组合在大多数表格任务上都是个非常稳健的强基线，调参也不敏感——“ $n\_estimators$  给大点、 $max\_features$  用  $\sqrt{d}$ ”基本就能跑出像样的结果。

## 5.3 Boosting：让后面的模型专攻前面的错题

现在换个完全不同的思路。Bagging 是“并行投票”，Boosting 则是“串行纠错”——一个模型先上，做完之后看看哪些样本预测错了，然后告诉下一个模型：“这几个你重点学”。一个接一个，每加一个模型都让整体的错误少一点。

最早的 Boosting 算法是 1995 年的 AdaBoost。它的做法很直接：每轮训练后，把**预测错的样本权重调高**，预测对的样本权重调低，让下一棵树不得不重视那些“难样本”。最后再把每棵树按“准确度”加一个权重投票，准的树话语权大。AdaBoost 从数学上回答了一个深刻的问题：**能不能把一堆“只比随机猜好一点点”的弱学习器，组合成强学习器？** 答案是肯定的——只要弱学习器的错误率严格小于  $1/2$ ，加到无限多个就能逼近 0 错误率。

但 AdaBoost 用的是指数损失，处理回归任务不直观。能不能换个角度：把 Boosting 写成**梯度下降**的形式？这就是**梯度提升树** (Gradient Boosted Decision Tree, GBDT) 的核心想法。

回顾一下梯度下降：要最小化损失  $L(\theta)$ ，就朝着负梯度方向走一小步， $\theta \leftarrow \theta - \eta \cdot \nabla L$ 。GBDT 把“参数  $\theta$ ”换成“整个模型  $F$ ”：当前模型是  $F(x)$ ，损失是  $L(y, F(x))$ ，下一步要更新的方向就是损失对  $F$  的**负梯度**。但  $F$  是一棵树，没法直接“沿梯度走”，于是让下一棵树去**拟合这个负梯度**——拟合上了，新模型加进来就等于让总损失下降了一步。

$$F_m(x) = F_{m-1}(x) + v \cdot h_m(x), h_m \approx -\partial L / \partial F$$

这里的  $r = -\partial L / \partial F$  被称为**伪残差**——它本质上就是“当前模型还差多少”，下一棵树专门去补这个差。 $v$  是学习率（也叫 shrinkage），让每棵树的贡献小一点，走很多小步通常比走少几大步更稳健。

GBDT 的妙处在于：把损失换一换就能做不同的任务。回归用平方损失，分类用对数损失，排序用 LambdaRank，生存分析用 Cox 损失——框架不变，只换损失函数即可。这种“以梯度为方向、以决策树为步”的形式，让 GBDT 成了表格数据上最灵活也最强大的算法之一。

## 5.4 XGBoost 与 LightGBM：从“GBDT 太慢”到工程优化

GBDT 原理虽好，但有两个工程痛点：一是**分裂查找太慢**——每次分裂都要遍历所有特征、所有候选切点；二是**容易过拟合**——加几百棵树很容易把训练集学到 100%。这两个痛点催生了两个明星算法：XGBoost 和 LightGBM。

**XGBoost** (2014) 的优化思路是"把损失展开到二阶": GBDT 只用了一阶导数(梯度), XGBoost 把损失做二阶泰勒展开, 同时用一阶  $g$  和二阶  $h$  两个量来更精确地估计分裂收益。牛顿法利用二阶导数信息(Hessian矩阵), 在每次迭代中同时考虑梯度和曲率, 因此收敛速度比一阶梯度下降更快。再加上正则项(叶节点数  $\gamma \cdot T$  + 叶值  $\frac{1}{2}\lambda \cdot ||w||^2$ )、列抽样、并行分裂查找、稀疏感知, XGBoost 在速度和精度上同时甩开了 GBDT, 一度统治了 Kaggle 的表格赛道。

**LightGBM** (2017, 微软) 则直接朝"分裂查找慢"这个最大瓶颈开刀。它的核心招数叫**直方图算法**: 把连续特征离散化到 255 个桶里, 分裂时不再遍历所有样本, 只遍历桶。复杂度从  $O(n \cdot d)$  降到  $O(b \cdot d)$ ,  $n$  是样本数(可能百万级),  $b$  是桶数(255), 速度提升几个量级。

LightGBM 还做了一个和 XGBoost 不同的生长策略: XGBoost 是**层优先**(level-wise, 一层一层长), LightGBM 是**叶子优先**(leaf-wise, 永远找当前损失下降最大的那个叶节点去分裂)。叶子优先长得更"贪", 相同叶节点数下精度更高, 但也更容易过拟合, 所以 LightGBM 一般要配 `max_depth` 或 `num_leaves` 来约束。

一句话总结: **XGBoost 用二阶展开换精度, LightGBM 用直方图换速度**。在大数据上 LightGBM 通常更快, 在精度上两者都接近上限——具体用哪个, 看数据规模和场景偏好。

## 5.5 算法实现描述: 随机森林的伪代码

讲到这里, 再回头看随机森林的"算法步骤"就会很顺。它就是前面三件事(数据随机 + 树 + 特征随机)的代码化, 没有更多新概念。

### 5.5.1 数据结构

- **trees**: 列表, 存  $T$  棵决策树
- **feature\_indices**: 每棵树使用的特征子集
- **参数**: `n_estimators` (树数  $T$ ), `max_depth`, `max_features` (特征子集大小  $m$ )

### 5.5.2 $\text{fit}(X, y)$ — 并行训练

**输入**: 数据  $(X, y)$ , 树数  $T$ , 特征子集大小  $m$  (通常  $\sqrt{d}$ )

**对  $t = 1, \dots, T$  并行:**

- 1. **Bootstrap 采样**: 从  $n$  个样本中有放回随机抽取  $n$  个 (允许重复)
- 2. **特征随机**: 从  $d$  个特征中随机选  $m$  个 ( $m \approx \sqrt{d}$ )
- 3. 在采样子集上训练一棵决策树 (不剪枝)
- 4. 保存树和特征索引

### 5.5.3 $\text{predict}(X)$ — 多数投票

对每个样本  $x$ : 让  $T$  棵树各自预测, 取众数作为最终结果:

$$\hat{y} = \text{mode}(\{h_t(x)\}_{t=1}^T)$$

### 5.5.4 复杂度

- **训练**:  $O(T \cdot n \cdot d \cdot \log n)$ , 可完全并行



· 预测： $O(T \cdot \log n)$ ，T 棵树并行投票

**[提示]** scikit-learn 用 joblib 并行训练，`n_jobs=-1` 利用全部 CPU。  
XGBoost/LightGBM 用直方图加速，单机即可处理千万级样本。

## 5.6 计算示例：手算一次 Bagging 投票

光讲原理容易飘，我们用一个最小例子把投票过程算一遍。假设训练集有 5 个二分类样本， $y = [1, 1, 1, 0, 0]$ 。现在建三棵树，每棵树做一次有放回采样：T1 抽到  $\{1, 2, 3, 3, 4\}$ ，T2 抽到  $\{2, 2, 3, 5, 5\}$ ，T3 抽到  $\{1, 1, 4, 4, 5\}$ 。因为每棵树学到的子集不一样，预测结果也不同：T1 输出  $[1, 1, 1, 1, 0]$ ，T2 输出  $[1, 1, 1, 0, 0]$ ，T3 输出  $[1, 1, 0, 0, 0]$ 。

看样本 4：T1 预测 1（错了！），T2 预测 0（对），T3 预测 0（对）。三棵树一投票，2 票对 0、1 票错 1——多数票是 0，正好和真实标签一致。再看样本 3：三棵树预测 1、1、0，多数票是 1，也对。这就是“一棵树错了但投票救回来”的直观体现。注意样本 4 在 T1 里其实**没出现**（T1 抽到的是  $\{1, 2, 3, 3, 4\}$ ，但样本 4 标签是 0，T1 学到的子集里样本 4 的标签也是 0，预测反而是 1——这正是过拟合噪声的典型表现），而 Bagging 通过多棵树叠加把它纠正了过来。

## 5.7 应用实例：特征重要性分析

集成树模型除了准，还有个意外的好处：**能告诉你哪些特征重要**。在金融风控、医疗诊断这类场景，业务方不光要预测准，还要知道“模型凭什么这么判断”——如果一个违约预测模型说“信用额度”重要，那它的预测就有业务依据；如果说“申请提交的小时数”重要，那大概率是数据泄漏或巧合，得警惕。

随机森林和 GBDT 都能输出特征重要性，常用度量有两种。一种叫**不纯度下降（MDI）**：决策树分裂时，每个特征带来的“信息增益”（不纯度下降量）都能算出来，把所有树所有节点累加一下，就是该特征的总贡献。它快、直观，但偏向高基数特征（取值多的特征天然占便宜）。另一种叫**置换精度（MDA）**：把某个特征的取值在测试集上随机打乱，看模型准确率掉多少——掉得越多说明这个特征越重要。它更稳健、有理论保证，但计算更贵。

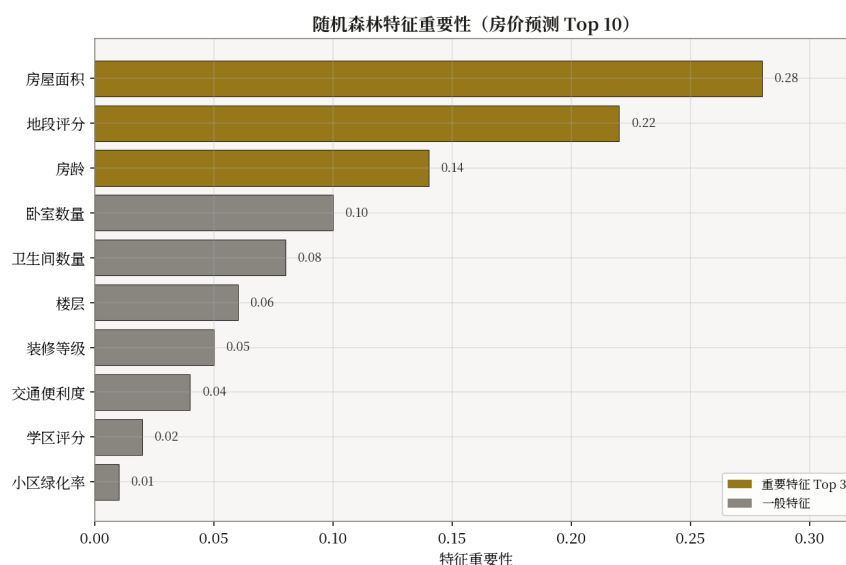


图 5.2 随机森林特征重要性条形图：账龄与信用额度是违约预测的关键驱动

在一份贷款违约数据集上的实验表明，特征“账龄”和“信用额度”的不纯度重要性占比分别达到 0.27 和 0.21，远高于其余变量。这与业务经验一致：账龄越长、额度越高，客户违约概率越低。所以特征重要性不仅能用来**解释模型**，还能用来**做特征选择**——把重要性排在前面的特征留下，把贡献接近 0 的剔除，既加速训练又减少噪声干扰。

## 5.8 代码实现：sklearn 与 XGBoost

原理讲完了，落到代码其实很简单。scikit-learn 给出了 RandomForest 和 GradientBoosting，XGBoost 则提供原生接口。下面这段代码同时演示两者，并提取特征重要性：

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import cross_val_score
import xgboost as xgb

X, y = load_breast_cancer(return_X_y=True)

# (1) 随机森林 + 5 折交叉验证
rf = RandomForestClassifier(n_estimators=300, max_features='sqrt',
                           oob_score=True, n_jobs=-1, random_state=42)
print(f'RF CV acc: {cross_val_score(rf, X, y, cv=5).mean():.4f}')

# 特征重要性
rf.fit(X, y)
importances = rf.feature_importances_
top5 = np.argsort(importances)[-5:][::-1]
print('Top-5 features:', top5)

# (2) XGBoost + 早停
from sklearn.model_selection import train_test_split
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.2, random_state=42)
dtr = xgb.DMatrix(X_tr, label=y_tr)
dva = xgb.DMatrix(X_va, label=y_va)
params = dict(objective='binary:logistic', eval_metric='auc',
              eta=0.05, max_depth=4, subsample=0.8,
              colsample_bytree=0.8, seed=42)
model = xgb.train(params, dtr, num_boost_round=2000,
                  evals=[(dtr, 'tr'), (dva, 'va')],
                  early_stopping_rounds=50, verbose_eval=200)

print(f'XGB best iter: {model.best_iteration}, AUC: {model.best_score:.4f}')
```

## 5.9 现代发展与小结

集成学习从 1990 年代被提出，到今天依然是表格数据建模的事实标准。一个直观的理由是：表格数据天生就是“特征 + 标签”，没有图像的空间结构、没有文本的时序结构，深度网络很难拿到比 GBDT 更好的精度，反而训练成本高得多。近年的新发展主要有几个方向：

**CatBoost** 通过目标编码（Ordered TS）天然处理类别特征，避免手工 one-hot；**NGBoost** 用自然梯度预测概率分布参数，输出不确定性；**TabNet / FT-Transformer** 把 Transformer 引入表格，在部分任务上超越 GBDT；**TabPFN** 等基于 in-context learning 的方法在小样本上展现出竞争力。

但在大多数结构化业务场景里，XGBoost / LightGBM 仍是首选基线——简单、快速、可解释、强稳定。理解了"为什么需要集成"和"三种组合思路"，你就掌握了集成学习的核心。

**[技巧]** 实践中表格建模的"三板斧": (1) 先用 LightGBM 跑通基线; (2) 用 Optuna 做贝叶斯调参; (3) 多个 GBDT + RF + 线性模型做 Stacking 提升最后 0.5 个百分点。

## 第六章 支持向量机

第二章线性回归画了一条线拟合数据。但如果任务是**分类**——比如区分垃圾邮件和正常邮件——我们需要的是一条**分界线**而不是拟合线。能分开两类的线有无数条，哪条最好？支持向量机（SVM）从这个问题出发，一步步推导出最大间隔、软间隔、hinge loss、核函数这一整套理论。本章的推导是一条连续的数学链，不是零散的概念罗列。

### 6.1 从直觉到优化问题：一步步推导

**出发点：**给定  $n$  个样本  $\{(x_i, y_i)\}$ ,  $y_i \in \{+1, -1\}$ ，找一个超平面  $w^T x + b = 0$  把两类分开。

**第一步：为什么"离两类都远"的线最好。**如果一条分界线离最近的正类点和最近的负类点都有较大的距离，那么新样本只要落点偏差超过这个距离，分类就不会错。反之如果线擦着某个点过，稍微一抖动就错分。因此"离两类都尽量远" = "对噪声鲁棒" = "泛化好"。

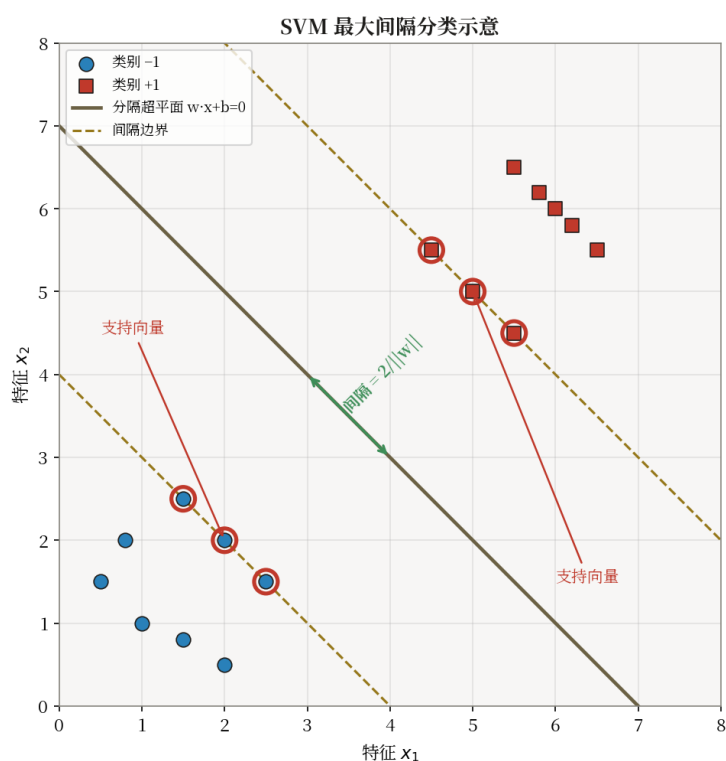


图 6.1 最大间隔超平面：粗实线为决策面，虚线为间隔边界，圈出的样本为支持向量

**第二步：量化"距离"。**点  $x$  到超平面  $w^T x + b = 0$  的几何距离为  $|w^T x + b| / \|w\|$ 。由于  $y_i$  的符号与类别一致，分类正确时  $y_i(w^T x_i + b) > 0$ ，所以距离可以去掉绝对值，写成  $y_i(w^T x_i + b) / \|w\|$ 。

**第三步：定义"间隔"。**我们要最大化"最近的点到超平面的距离"，即：

$$\max_{w, b} \min_i y_i (w^T x_i + b) / \|w\|$$

**第四步：消除尺度自由度。** $w$  和  $b$  可以同时乘以任意正数而不改变超平面位置 ( $w^T x + b = 0$  和  $2w^T x + 2b = 0$  是同一个平面)。利用这个自由度，设最近样本满足  $y_i(w^T x_i + b) = 1$  (

即把  $w, b$  缩放到使最近点恰好为 1)。于是间隔  $= 2 / ||w||$  (正类最近点 +1、负类最近点 -1, 之间距离  $2/||w||$ )。

**第五步：转化为优化问题。**最大化  $2/||w||$  等价于最小化  $||w||^2/2$ , 加上所有样本分类正确的约束:

$$\min_{w,b} \frac{1}{2} ||w||^2 \text{ s.t. } y_i(w^T x_i + b) \geq 1, \text{ for all } i$$

这就是**硬间隔 SVM**——一个凸二次规划问题。解出后会发现只有少数样本满足等号成立 ( $y_i(w^T x_i + b) = 1$ )，这些“踩在间隔边界上”的样本就是**支持向量**——它们撑住了决策面的位置。移动非支持向量不影响结果，移动支持向量则决策面跟着变。

## 6.2 软间隔：从约束到 hinge loss

上面的优化问题要求所有样本满足  $y_i(w^T x_i + b) \geq 1$ 。但现实中几乎总有噪声点跨越边界，硬约束会导致无解或决策面被拉歪。解决思路：**允许少量样本越界，但要付出代价**。

**第六步：引入松弛变量。**为每个样本引入  $\xi_i \geq 0$ ，表示越界程度。把约束从  $y_i(w^T x_i + b) \geq 1$  放宽为  $y_i(w^T x_i + b) \geq 1 - \xi_i$ ，目标函数加上惩罚项  $C \cdot \sum \xi_i$ :

$$\min_{w,b,\xi} \frac{1}{2} ||w||^2 + C \cdot \sum_i \xi_i \text{ s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

$C$  控制越界代价:  $C$  大  $\rightarrow$  尽量不越界  $\rightarrow$  接近硬间隔  $\rightarrow$  易过拟合;  $C$  小  $\rightarrow$  允许越界  $\rightarrow$  间隔更宽  $\rightarrow$  更平滑  $\rightarrow$  易欠拟合。

**第七步：消去松弛变量，得到无约束损失函数。**由于目标函数中  $\xi_i$  前系数  $C > 0$ ，要让目标最小， $\xi_i$  会取约束允许的最小值:

- 当样本满足间隔 ( $y_i(w^T x_i + b) \geq 1$ ) 时， $\xi_i = 0$  即可满足约束。
- 当样本违反间隔 ( $y_i(w^T x_i + b) < 1$ ) 时，需取  $\xi_i = 1 - y_i(w^T x_i + b) > 0$ 。

两种情况统一写成:

$$\xi_i^*(w, b) = \max(0, 1 - y_i(w^T x_i + b))$$

回代到目标函数，得到**无约束形式**:

$$L(w, b) = \frac{1}{2} ||w||^2 + C \cdot \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b))$$

第一项  $\frac{1}{2} ||w||^2$  是 L2 正则项; 第二项就是**hinge loss** (合页损失) ——函数  $f(t) = \max(0, 1-t)$  在  $t \geq 1$  时取 0 (满足间隔不罚)，在  $t < 1$  时线性增长 (违反越远罚越重)，在  $t = 1$  处有一个折角。这个折角是 hinge loss 的本质特征，也是下一步分情况求导的根源。

## 6.3 梯度推导：从 hinge loss 到更新规则

有了无约束损失  $L(w, b)$ ，就可以用梯度下降优化。下面一步步求  $L$  对  $w$  和  $b$  的梯度。

**第八步：对  $w$  求偏导——拆成两项。**由导数线性性， $\partial L / \partial w = \partial (\frac{1}{2} ||w||^2) / \partial w + \partial (C \cdot \sum \max(\dots)) / \partial w$ 。

**第一项：**  $\frac{1}{2}||w||^2 = \frac{1}{2} \cdot w^T w$ ，对  $w$  求梯度得  $w$ （由  $\partial(x^T x)/\partial x = 2x$  再乘  $\frac{1}{2}$ ）。

$$\partial(\frac{1}{2} ||w||^2) / \partial w = w$$

**第二项：** 对每个样本  $i$ ，记  $m_i = 1 - y_i(w^T x_i + b)$ ，该样本的 hinge 项为  $C \cdot \max(0, m_i)$ 。  
由链式法则：

$$\partial [C \cdot \max(0, m_i)] / \partial w = C \cdot \partial \max(0, m_i) / \partial m_i \cdot \partial m_i / \partial w$$

其中  $\partial m_i / \partial w$ ：  $m_i$  中含  $w$  的部分是  $-y_i \cdot w^T x_i$ ，对  $w$  求梯度得  $-y_i \cdot x_i$ 。

**第九步：分情况讨论  $\max(0, m)$  的导数。** 函数  $\max(0, m)$  在  $m > 0$  时取  $m$ （导数为 1），在  $m < 0$  时取 0（导数为 0），在  $m = 0$  处不可导——这是 hinge loss 折角的数学体现。将  $m_i$  与是否违反间隔对应：

- **情况 A：违反间隔**，即  $m_i > 0 \Leftrightarrow y_i(w^T x_i + b) < 1$ 。此时  $\max(0, m_i) = m_i$ ， $\partial \max / \partial m_i = 1$ 。合起来： $\partial [C \cdot \max] / \partial w = C \cdot 1 \cdot (-y_i \cdot x_i) = -C \cdot y_i \cdot x_i$ 。
- **情况 B：满足间隔**，即  $m_i \leq 0 \Leftrightarrow y_i(w^T x_i + b) \geq 1$ 。此时  $\max(0, m_i) = 0$  是常数，导数为 0，该样本的 hinge 项对  $w$  没有梯度贡献。

在  $m_i = 0$ （恰好踩在间隔边界）处不可导，这正是**支持向量**所在的位置——它们是让间隔被撑开的关键样本。

**第十步：综合得到  $\partial L / \partial w$ 。** 把两项相加，用指示函数 [违反 <sub>$i$</sub> ]（违反时取 1，否则取 0）：

$$\partial L / \partial w = w - C \cdot \sum_i [\text{违反}_i] \cdot y_i \cdot x_i$$

**第十一步：对  $b$  求偏导。** 第一项  $\frac{1}{2}||w||^2$  不含  $b$ ，对  $b$  求导为 0。第二项中  $\partial m_i / \partial b = -y_i$ （因为  $m_i$  中含  $b$  的部分是  $-y_i \cdot b$ ）。同理分情况：

$$\partial L / \partial b = -C \cdot \sum_i [\text{违反}_i] \cdot y_i$$

**第十二步：SGD 更新规则。** 采用随机梯度下降（每次取一个样本更新），将上述梯度拆成单样本形式：

- **满足间隔** ( $y_i(w^T x_i + b) \geq 1$ )：[违反 <sub>$i$</sub> ] = 0，hinge 项梯度为 0，只剩正则项。 $\nabla_w = w$ ， $\nabla_b = 0 \rightarrow w \leftarrow w - \alpha \cdot w$ ， $b$  不变
- **违反间隔** ( $y_i(w^T x_i + b) < 1$ )：[违反 <sub>$i$</sub> ] = 1，hinge 项梯度为  $-C \cdot y_i \cdot x_i$ 。 $\nabla_w = w - C \cdot y_i \cdot x_i$ ， $\nabla_b = -C \cdot y_i \rightarrow w \leftarrow w - \alpha \cdot (w - C \cdot y_i \cdot x_i)$ ， $b \leftarrow b + \alpha \cdot C \cdot y_i$

## 6.4 核函数：当数据线性不可分时

前面的推导都假设数据可以用超平面分开（线性可分）。但如果数据结构本身是非线性的——比如月牙形和圆形交错——再怎么调  $C$  都画不出一条直线。

**思路：** 把数据映射到更高维空间，在高维中找超平面。例如把 2D 点  $(x_1, x_2)$  映射为 3D 点  $(x_1, x_2, x_1^2 + x_2^2)$ ，在高维空间中原本不可分的数据可能变得线性可分。



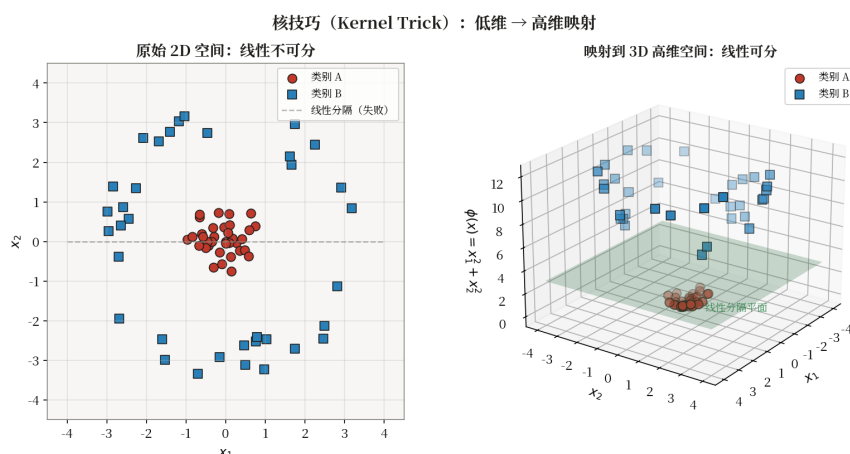


图 6.2 核技巧示意：原始 2D 空间线性不可分，映射到高维后变得线性可分

**问题：**显式映射  $\phi(x)$  维度可能极高（甚至无穷），计算  $\phi(x_i) \cdot \phi(x_j)$  代价爆炸。

**核技巧：**SVM 的对偶形式中所有计算只涉及样本间内积  $x_i \cdot x_j$ ，不直接出现  $\phi(x)$ 。因此只需定义核函数  $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ ，直接给出高维空间的内积，无需显式映射。

从拉格朗日对偶推导（引入乘子  $\alpha_i \geq 0$ ，对  $w$  和  $b$  求偏导令其为零），得到对偶问题：

$$\max_{\alpha} \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

约束  $0 \leq \alpha_i \leq C$  且  $\sum \alpha_i y_i = 0$ 。决策函数  $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$ 。

常见核函数：

- **线性核：**  $K(x, z) = x \cdot z$ ，等价于不映射。
- **RBF / 高斯核：**  $K(x, z) = \exp(-\gamma \cdot \|x - z\|^2)$ ，最常用，可逼近任意连续边界。
- **多项式核：**  $K(x, z) = (\gamma \cdot x \cdot z + r)^d$ 。

## 6.7 算法实现描述

前面 6.4 节已推导出软间隔 SVM 的无约束损失函数  $L = \frac{1}{2} \|w\|^2 + C \cdot \sum \max(0, 1 - y_i(w^T x_i + b))$ ，并分情况求导得到了梯度更新规则。本节将这些结论整理为伪代码。

### 6.7.1 数据结构

- **权重  $w$ ：** 向量  $[d]$ ，超平面法向量
- **偏置  $b$ ：** 标量
- **参数：**  $C$ （正则化系数），学习率  $\alpha$ ，迭代次数  $T$

### 6.7.2 $\text{fit}(X, y) - \text{SGD 更新}$

**输入：**  $X \in \mathbb{R}^{n \times d}$ ， $y \in \{-1, +1\}^n$

**初始化：**  $w \leftarrow 0, b \leftarrow 0$

**对每个  $\text{epoch} = 1, \dots, T$ ，遍历每个样本  $(x_i, y_i)$ ：**

**步骤 1：** 计算间隔  $\text{margin} = y_i(w^T x_i + b)$

- 若  $\text{margin} \geq 1$  (满足间隔) :  $\nabla_w = w, \nabla_b = 0 \rightarrow w \leftarrow w - \alpha \cdot w, b$  不变
- 若  $\text{margin} < 1$  (违反间隔) :  $\nabla_w = w - C \cdot y_i \cdot x_i, \nabla_b = -C \cdot y_i \rightarrow w \leftarrow w - \alpha \cdot (w - C \cdot y_i \cdot x_i), b \leftarrow b + \alpha \cdot C \cdot y_i$

上述规则的数学推导见 6.4 节: hinge loss  $\max(0, 1 - \text{margin})$  在  $\text{margin} \geq 1$  时梯度为 0, 在  $\text{margin} < 1$  时梯度为  $-y_i \cdot x_i$ , 与 L2 正则项的梯度  $w$  相加即得。

### 6.7.3 predict(X) 与 decision\_function(X)

$$\hat{y} = \text{sign}(w^T x + b)$$

decision\_function 返回到超平面的有符号距离  $f(x) = w^T x + b$ , 可用于排序或提取置信度。

### 6.7.4 复杂度

- 梯度下降训练:  $O(ndT)$ , 每次遍历全部样本
- SMO 算法:  $O(n^2d) \sim O(n^3)$ , 更精确
- 预测:  $O(d)$ , 单次内积

**[提示]** scikit-learn 的 SVC 用 libsvm (SMO 算法), 支持各种核函数; LinearSVC 用线性规划, 速度快。

## 6.8 计算示例: 手算 2D SVM 间隔

原理讲完来手算一个最小例子。设 2D 平面上 4 个样本: 正类 (2,2)、(3,3), 负类 (0,0)、(-1,-1)。它们线性可分, 乍看决策面应该是  $y = x$  这条对角线 ( $w = (1,-1)/\sqrt{2}$ 、 $b = 0$ )。

但稍微一算就发现  $y = x$  不行: 点 (2,2) 到  $y = x$  的距离是  $|2-2|/\sqrt{2} = 0$ , 点 (0,0) 到  $y = x$  的距离也是 0——这条线穿过样本本身, 间隔为 0, 显然不是最大间隔解。我们需要把决策面平移一下, 让它"夹"在两类之间。

试  $w = (1,-1)$ 、 $b = -1$ , 决策面是  $x_1 - x_2 = 1$  (仍平行于  $y = x$ , 但平移过)。正类 (2,2):  $|2-2-1|/\sqrt{2} = 1/\sqrt{2}$ ; 正类 (3,3):  $|3-3-1|/\sqrt{2} = 1/\sqrt{2}$ ; 负类 (0,0):  $|0-0-1|/\sqrt{2} = 1/\sqrt{2}$ ; 负类 (-1,-1):  $|-1+1-1|/\sqrt{2} = 1/\sqrt{2}$ 。所有点到决策面的距离都等于  $1/\sqrt{2}$ , 意味着四个点都在间隔边界上——都是支持向量。间隔  $= 2 \times (1/\sqrt{2}) = \sqrt{2} \approx 1.41$ 。这就是最大间隔的几何含义。

## 6.9 应用实例: 手写数字识别

在 MNIST 手写数字数据集上, SVM 长期是经典基线。使用 RBF 核、 $C=10$ 、 $\gamma=0.01$ , 能在 6 万训练样本上取得约 98.5% 的测试准确率, 与浅层 MLP 相当, 但训练时间远小于同等精度的深度网络。对小型 OCR、人脸识别、医学影像分类等任务, SVM 因"小数据也能跑得动、泛化稳健"的特点, 至今仍是首选工具之一。

## 6.10 代码实现: sklearn SVC

scikit-learn 提供了 SVC 类, 调用很简单。下面这段代码演示如何用 Pipeline 串联标准化和 SVM, 并做网格搜索调  $C$  和  $\gamma$ :

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
stratify=y, random_state=42)

pipe = Pipeline([('sc', StandardScaler()), ('svm', SVC())])

param_grid = [
    {'svm__kernel': ['linear'], 'svm__C': [0.1, 1, 10]},
    {'svm__kernel': ['rbf'], 'svm__C': [1, 10, 100],
     'svm__gamma': [1e-3, 1e-2, 1e-1]},
]
gs = GridSearchCV(pipe, param_grid, cv=3, scoring='accuracy',
n_jobs=-1, verbose=1)
gs.fit(X_tr, y_tr)
print('Best params:', gs.best_params_)
print(f'Best CV acc: {gs.best_score_:.4f}')
y_pred = gs.predict(X_te)
print(classification_report(y_te, y_pred))
```

## 6.11 现代发展与小结

SVM 的优美理论影响了后续许多算法。但在大数据时代，SVM 的二次规划求解复杂度  $O(n^2 \sim n^3)$  成了瓶颈——上百万样本就跑不动了，业界逐渐转向 GBDT 与深度网络。不过 SVM 仍以独特优势保留一席之地：**核 SVM** 在小样本、非线性问题上表现稳健；**深度核学习**（Deep Kernel Learning）将深度网络作为可学习特征提取器后接核 SVM，兼顾表示力与泛化；**One-Class SVM** 用于异常检测，**结构化 SVM** 处理结构化输出。

回顾这一章，从“哪条分界线最好”出发，我们一步步推出了最大间隔、支持向量、软间隔、核函数这一整套理论。理解了 SVM，就理解了正则化、对偶、核方法的最佳入口——这些概念在后续章节会反复出现。

**[技巧]** 调参经验：先尝试 RBF 核 + 网格  $\{C: [10^k, k \in [-2, 4]], \gamma: [10^k, k \in [-5, 1]]\}$ ，按对数尺度做 5 折交叉验证。若 RBF 远不及线性核，多半是数据本就接近线性，此时改用线性 SVM（sklearn LinearSVC）可大幅加速。

## 第七章 神经网络基础

到目前为止我们学的模型——线性回归、SVM、决策树——本质上都是一个**函数**。线性回归是一个线性函数加个损失；SVM 是一个超平面加间隔约束；决策树是分段常数函数。它们在简单问题上很好用，但遇到图像、语音、文本这类高维结构化数据就力不从心了。

为什么？因为现实问题太复杂，一个简单的函数表达不出来。比如识别一张  $28 \times 28$  的手写数字图像，输入是 784 维像素，输出是 0-9 这 10 个类别——这中间的映射关系不是一条线或几条切分能描述的。那能不能用**很多函数叠在一起**，组成一个更复杂的函数？这就是神经网络的基本出发点。

这一章我们就从“一个神经元长什么样”开始，一步步推出多层感知机、前向传播、反向传播，最后落到 MNIST 这个经典任务上。所有现代深度学习——CNN、RNN、Transformer、大语言模型——底子都是这一套，差别只在“怎么把神经元连起来”。

### 7.1 神经元：加权投票 + 决策

神经网络最小的零件叫**神经元**（neuron），它的设计直接模仿了生物神经元：多个输入信号各占一定权重，加起来超过某个阈值就“激活”，否则就“沉默”。换句话说，一个神经元就是一次**加权投票 + 决策**。

具体写出来：输入向量  $x \in \mathbb{R}^d$ ，权重向量  $w \in \mathbb{R}^d$ ，偏置标量  $b$ ，先算加权和  $z = w \cdot x + b$ ，再过一个非线性激活函数  $\sigma$ ，输出  $a = \sigma(z)$ 。这里的“加权投票”是  $w \cdot x + b$  这一步，“决策”是  $\sigma$  这一步。 $\sigma$  通常是个非线性的“门”——比如 Sigmoid 把  $z$  压到  $(0,1)$ ，ReLU 把负值截掉只保留正值。

早在 1958 年 Rosenblatt 就提出了最早的神经元形式——**感知机**，用阶跃函数  $\text{sign}(\cdot)$  做激活。它能学一些简单的线性分类，但 1969 年 Minsky 证明单层感知机连最简单的“异或”（XOR）都学不会，这一打击直接导致了神经网络第一次“寒冬”。问题不在于神经元本身，而在于“只有一个神经元 + 阶跃激活”组合太弱。

### 7.2 把单个神经元串起来：多层感知机

单层感知机搞不定 XOR，怎么救？思路很自然：**多堆几层**。一层算一组特征，下一层在上一层算出来的特征上再做一次加权投票。这就是**多层感知机**（Multi-Layer Perceptron, MLP）。

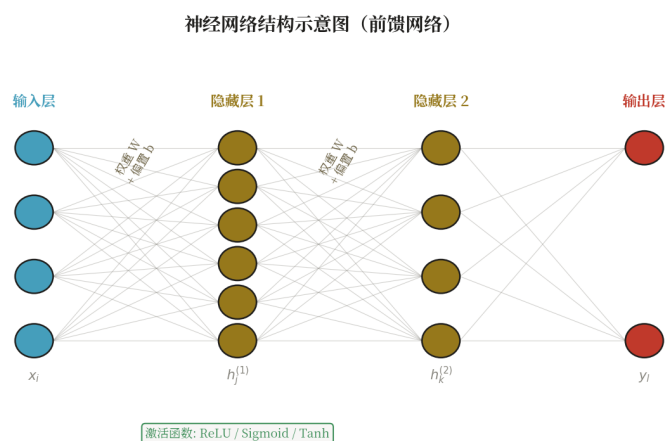


图 7.1 多层感知机结构：输入层 → 隐藏层（带激活） → 输出层

一个  $L$  层 MLP 可以写成递推式： $a_0 = x$ （输入）； $a_1 = \sigma_1(W_1 \cdot a_{0-1} + b_1)$ （第 1 层输出）；最后  $a_L = \hat{y}$ （输出）。中间的层叫**隐藏层**，因为它不直接对应输入也不直接对应输出，只是"偷偷"在中间算了一组特征。

为什么多层就能搞定 XOR？因为单层神经元只能在输入空间画一条直线，而 XOR 的两条分界是两条交叉的直线，一条画不出来。但两层 MLP 可以：第一层画两条直线把空间切成几块，第二层再把这些块重新组合一下，就能表达 XOR。理论上甚至有**万能逼近定理**：只要隐藏层足够宽，一个两层 MLP 可以逼近任意连续函数。

输出层根据任务选激活函数：回归用线性输出（不加激活）；二分类用 Sigmoid（输出概率）；多分类用 Softmax（输出 10 个类别的概率分布）。

## 7.3 怎么训练：前向传播 + 反向传播

现在网络搭好了，参数  $W$  和  $b$  全是随机初始化的，啥也不会。怎么让它们"学会"识别数字？这就回到了第二章线性回归的老套路：**定义损失函数衡量预测与真实的差距，然后用梯度下降更新参数**。唯一的问题在于：神经网络有多层参数，每层都要算梯度，怎么算得过来？

答案分两步。**前向传播**（forward pass）：信号从输入开始，一层一层往后算到输出，得到预测  $\hat{y}$ ，并算出损失  $L(y, \hat{y})$ 。这一步是"沿着网络走一遍"，把每一层的中间结果  $z$  和  $a$  都存下来。**反向传播**（backpropagation）：从损失出发，用**链式法则**把梯度从输出层往输入层一层一层倒推回来，得到每一层参数的梯度  $\partial L / \partial W$  和  $\partial L / \partial b$ 。

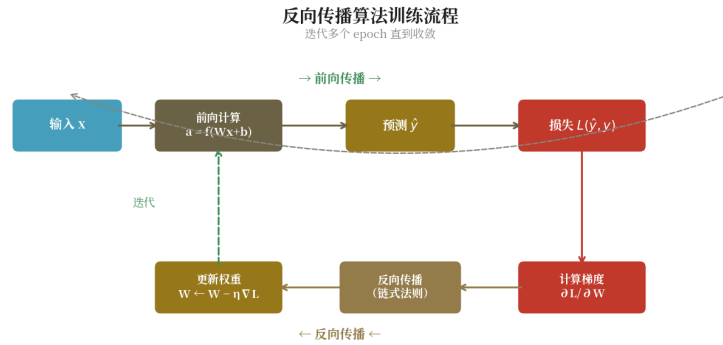


图 7.2 反向传播：从输出层向输入层逐层回传梯度

## 7.4 反向传播的链式推导：从输出层一路倒推回来

反向传播听起来神秘，本质上只有两件事：**链式法则 + 矩阵运算**。为了让推导清楚，先把符号约定写齐：第 1 层先做线性变换  $z_1 = W_1 \cdot a_{l-1} + b_1$ ，再过激活函数得到  $a_1 = \sigma_1(z_1)$ ，其中  $W_1$  是权重矩阵， $b_1$  是偏置向量， $\sigma_1$  是该层激活函数；第 L 层是输出层 ( $a_L = \hat{y}$ )，第 0 层是输入 ( $a_0 = x$ )。反向传播的目的是求出每一层的  $\partial L / \partial W_1$  和  $\partial L / \partial b_1$ 。为此先定义一个核心量——误差信号  $\delta_l \triangleq \partial L / \partial z_l$ ，也就是“损失对第 l 层预激活值的梯度”。只要把每一层的  $\delta_l$  算清楚，权重和偏置的梯度就能直接从  $\delta_l$  推出来。

**第一步：计算输出层误差  $\delta_L$ 。**输出层先做线性变换  $z_L = W_L \cdot a_{L-1} + b_L$ ，再经激活得到  $a_L = \sigma_L(z_L) = \hat{y}$ ，损失 L 依赖于  $a_L$ 。由链式法则  $z_L \rightarrow a_L \rightarrow L$  两段复合，有：

$$\delta_L = \partial L / \partial z_L = (\partial L / \partial a_L) \odot \sigma'_L(z_L)$$

其中  $\odot$  表示按元素相乘（Hadamard 积）。第一项  $\partial L / \partial a_L$  由损失函数形式决定：若 L 是均方误差，则  $\partial L / \partial a_L = a_L - y$ ；若输出配合 Softmax 用交叉熵，Softmax 的雅可比与交叉熵的梯度相乘恰好抵消，最终  $\partial L / \partial a_L$  也化简为  $a_L - y$ ——这是 Softmax + CE 在工程上几乎成标配的关键原因。第二项  $\sigma'_L(z_L)$  由激活函数本身决定，链式至此就停在了输出层。

**第二步：误差从后一层倒推回前一层，递推得到  $\delta_1$ 。**有了  $\delta_{l+1} = \partial L / \partial z_{l+1}$ ，要算  $\delta_l = \partial L / \partial z_l$ 。注意  $z_l$  影响 L 的路径只有一条： $z_l \rightarrow a_l \rightarrow z_{l+1} \rightarrow L$ ，其中  $z_{l+1} = W_{l+1} \cdot a_l + b_{l+1}$  对  $a_l$  求导得  $W_{l+1}$ ， $a_l = \sigma_l(z_l)$  对  $z_l$  求导得  $\sigma'_l(z_l)$ 。把这两段链式法则合起来：

$$\delta_l = (W_{l+1})^T \cdot \delta_{l+1} \odot \sigma'_l(z_l)$$

读法很直接：把后一层的误差信号  $\delta_{l+1}$  通过权重矩阵的转置  $(W_{l+1})^T$  “倒着投影”回本层，得到的是  $\partial L / \partial a_l$ ；再按元素乘上本层激活函数的导数  $\sigma'_l(z_l)$ ，就把梯度从激活端“折回”到预激活端，得到  $\delta_l$ 。这一递推关系与层数无关，从  $l = L-1$  一路用到  $l = 1$ ，递推 L-1 次即可得到所有隐藏层的误差信号——这也是“反向”二字的由来：信号沿着网络从输出层倒流回输入层。

**第三步：由误差信号算权重与偏置的梯度。**回到线性变换  $z_l = W_l \cdot a_{l-1} + b_l$ 。对  $W_l$  而言， $z_l$  是  $a_{l-1}$  的线性组合，故  $\partial z_l / \partial W_l = a_{l-1}$ ；对  $b_l$  而言， $\partial z_l / \partial b_l = 1$ （向量）。再乘上  $\delta_l = \partial L / \partial z_l$  即得：



$$\partial L / \partial W_1 = \delta_1 \cdot (a_{1-1})^T, \partial L / \partial b_1 = \delta_1$$

权重梯度是一个外积： $\delta_1$ （本层误差）与  $(a_{1-1})^T$ （上一层激活的转置）做矩阵乘积。这意味着某条连接的权重是否需要"大改"，由两端的的活动共同决定——若上游激活  $a_{1-1}$  强、下游误差  $\delta_1$  也大，该连接的更新幅度就大；反之就小。偏置梯度等于误差信号本身，因为偏置对预激活的偏导恒为 1，没有任何缩放。

把这三步合起来，反向传播算法的全貌就清晰了：前向计算各层的  $z_l$  与  $a_l$  并缓存；由输出层算出  $\delta_L$ ，再用  $\delta_l = (W_{l+1})^T \cdot \delta_{l+1} \odot \sigma'_l(z_l)$  逐层倒推到输入层；最后用  $\partial L / \partial W_l = \delta_l \cdot (a_{l-1})^T$ 、 $\partial L / \partial b_l = \delta_l$  取出每层参数的梯度。一次前向 + 一次反向，就完成了 mini-batch 上所有参数的梯度计算；随后用梯度下降  $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$  更新参数，这就是神经网络一次迭代的完整过程。现代深度学习框架（PyTorch、TensorFlow）都内置自动微分（autograd），只要定义前向计算，框架就会自动按这套链式法则跑反向传播——但亲手理解这套推导，是看懂框架内部行为、调试训练问题的前提。

## 7.5 激活函数：没有非线性，多层等于一层

激活函数为什么需要？这是初学者最容易忽略的一个关键问题。答案非常直接：**如果没有激活函数，多层 MLP 数学上等价于一层线性模型。**

我们来证一下：假设三层都不加激活，每一层都是  $a_l = W_l \cdot a_{l-1} + b_l$ 。那么  $a_3 = W_3 \cdot (W_2 \cdot (W_1 \cdot x + b_1) + b_2) + b_3$ ，展开就是  $(W_3 W_2 W_1) \cdot x + (W_3 W_2 b_1 + W_3 b_2 + b_3)$ ，仍然是  $x$  的线性函数。换句话说，**线性组合的线性组合还是线性**，堆再多隐藏层也表达不出 XOR 这种非线性关系。

加上激活函数  $\sigma$  就打破了这种"线性塌缩"——每过一层都引入一个非线性"扭折"，整体函数就不再是线性的了。这就是激活函数的存在意义。常用的激活函数有：

- **Sigmoid**:  $\sigma(z) = 1/(1+e^{-z})$ ，输出  $\in (0,1)$ ，历史最早；但梯度易饱和（两端导数接近 0），深层网络中常导致梯度消失。
- **Tanh**:  $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$ ，输出  $\in (-1,1)$ ，零中心化但仍饱和。
- **ReLU**:  $\max(0, z)$ ，计算简单、缓解梯度消失；但负半轴梯度为 0，可能造成"神经元死亡"。
- **Leaky ReLU / GELU / Swish**: ReLU 改进，兼顾稀疏性与平滑性，广泛用于现代深度学习网络。

## 7.6 为什么 ReLU 比 Sigmoid 好：梯度消失问题

上面提到 Sigmoid 在深层网络中"梯度消失"，这是什么意思？看反向传播公式  $\delta_l = (W_{l+1})^T \cdot \delta_{l+1} \odot \sigma'_l(z_l)$ 。每往回传一层，都要乘一次激活函数的导数  $\sigma'(z)$ 。Sigmoid 的导数  $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$ ，最大值只有 0.25（ $z=0$  处），两端迅速衰减到接近 0。

于是在一个 10 层的 Sigmoid 网络里，从输出层倒推回输入层要乘 10 次小于 0.25 的数，梯度以指数速度衰减——到输入层时几乎为 0，前面几层的权重根本没法更新。这就是**梯度消失**，是早期深网络训不动的核心原因。

**ReLU** 正是为此而生。ReLU 的导数在正半轴恒等于 1，负半轴为 0。只要神经元激活 ( $z > 0$ )，梯度反向传播时就能"原样"传过去，不被乘小——这从根上缓解了梯度消失。加上 ReLU 计算简单（一个 max 操作）、让网络稀疏（一半神经元输出 0），它成了现代深度网络的主力激活。

当然 ReLU 也有自己的毛病——"神经元死亡"：如果某次更新后某个神经元的权重让  $z$  永远小于 0，那它后续梯度永远为 0，再也不会被更新，相当于"死了"。Leaky ReLU（负半轴给一个小斜率如 0.01x）和 GELU 就是针对这个问题的改进。

## 7.7 损失函数与优化器

神经网络训练离不开两件事：用什么损失衡量"差多少"，用什么优化器更新参数。损失函数按任务选：

- **均方误差 MSE**:  $L = (1/n)\sum(y-\hat{y})^2$ ，用于回归；对异常值敏感。
- **二元交叉熵 BCE**:  $L = -\sum[y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y})]$ ，用于二分类。
- **多类交叉熵 CE**:  $L = -\sum y_k \cdot \log \hat{y}_k$ ，配合 Softmax 输出，是多分类标准损失。
- **加 L2 正则**:  $L_{\text{total}} = L + \lambda \cdot \|W\|^2$ ，防过拟合；Dropout、BatchNorm 也起正则作用。

优化器里最朴素的是**SGD**（小批量梯度下降），但收敛慢、容易在峡谷地形里来回震荡。

**Momentum** 累积历史梯度方向，相当于给优化加了"惯性"，减少震荡： $v = \beta \cdot v - \eta \cdot \nabla$ ， $\theta \leftarrow \theta + v$ 。**Adam** 同时维护一阶动量  $m$  和二阶动量  $v$  并做偏差修正，是当前最常用的优化器：

$$m = \beta_1 \cdot m + (1-\beta_1) \cdot g; v = \beta_2 \cdot v + (1-\beta_2) \cdot g^2; \theta \leftarrow \theta - \eta \cdot m / \sqrt{v}$$

默认  $\beta_1=0.9$ ,  $\beta_2=0.999$ ，学习率  $\eta$  在训练中常用 warmup + cosine 衰减。Adam 几乎是"开箱即用"的优化器，新手项目先用它基本不会错。

## 7.8 算法实现描述

7.4 节已推导出反向传播的链式法则。本节将其整理为伪代码。

### 7.8.1 数据结构

- $W_1 \in \mathbb{R}^{d \times h}$ （输入→隐藏层权重），He 初始化
- $b_1 \in \mathbb{R}^h$ （隐藏层偏置）
- $W_2 \in \mathbb{R}^{h \times c}$ （隐藏→输出层权重）
- $b_2 \in \mathbb{R}^c$ （输出层偏置）
- **参数**：学习率  $\alpha$ , 迭代次数  $E$

### 7.8.2 fit(X, y) — 前向 + 反向

**输入**:  $X \in \mathbb{R}^{n \times d}$ , one-hot 标签  $Y \in \{0,1\}^{n \times c}$

**对每个 epoch**:

**前向传播**（7.3 节推导）:

$$z_1 = X \cdot W_1 + b_1 [n, h]$$

- $a_1 = \text{ReLU}(z_1) = \max(0, z_1)$  [n, h]
- $z_2 = a_1 \cdot W_2 + b_2$  [n, c]
- $p = \text{Softmax}(z_2)$ :  $p_{ij} = \exp(z_{2ij}) / \sum_k \exp(z_{2ik})$

损失（交叉熵）：  $L = -(1/n) \cdot \sum_i \log p_{i,y_i}$

反向传播（7.4 节链式法则推导的结果）：

- $\partial L / \partial z_2 = (p - Y) / n$  [n, c] (softmax + 交叉熵的合并梯度)
- $\partial L / \partial W_2 = a_1^T \cdot \partial L / \partial z_2$  [h, c]
- $\partial L / \partial b_2 = \sum \partial L / \partial z_2$  [c]
- $\partial L / \partial a_1 = \partial L / \partial z_2 \cdot W_2^T$  [n, h] (误差回传到隐藏层)
- $\partial L / \partial z_1 = \partial L / \partial a_1 \odot (z_1 > 0)$  [n, h] (ReLU 导数:  $z_1 > 0$  时为 1, 否则为 0)
- $\partial L / \partial W_1 = X^T \cdot \partial L / \partial z_1$  [d, h]
- $\partial L / \partial b_1 = \sum \partial L / \partial z_1$  [h]

参数更新 (SGD)：

- $W_1 \leftarrow W_1 - \alpha \cdot \partial L / \partial W_1, b_1 \leftarrow b_1 - \alpha \cdot \partial L / \partial b_1$
- $W_2 \leftarrow W_2 - \alpha \cdot \partial L / \partial W_2, b_2 \leftarrow b_2 - \alpha \cdot \partial L / \partial b_2$

### 7.8.3 predict(X) 与复杂度

前向传播到  $z_2$ , 返回  $\text{argmax}(z_2, \text{axis}=1)$

- 训练:  $O(n \cdot (d \cdot h + h \cdot c) \cdot E)$ ,  $h$  为隐藏层宽度,  $E$  为 epoch
- 预测:  $O(d \cdot h + h \cdot c)$ , 两次矩阵乘法

**[提示]** PyTorch 的 autograd 自动计算梯度, 无需手动写反向传播。本伪代码展示"反向传播 = 链式法则 + 矩阵运算"的本质。

## 7.9 计算示例：2 层网络的前向 + 反向手算

原理有了, 来手算一次。设 2 层 MLP: 输入  $x = [1, 2]$ , 第一层  $W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ ,  $b_1 = [0, 0]$ , 激活 ReLU; 第二层  $W_2 = [1, -1]$ ,  $b_2 = 0.5$ , 输出标量  $\hat{y}$ 。

**前向:**  $z_1 = W_1 \cdot x + b_1 = [1, 2]$ ;  $a_1 = \text{ReLU}(z_1) = [1, 2]$  (都为正, 不变);  $z_2 = W_2 \cdot a_1 + b_2 = 1 \cdot 1 + (-1) \cdot 2 + 0.5 = -0.5$ ;  $\hat{y} = z_2 = -0.5$  (线性输出)。

**损失:** 若任务为回归、损失为  $(y - \hat{y})^2$  且真实  $y = 1$ , 则  $L = (1 - (-0.5))^2 = 2.25$ 。

**反向 (一步一步倒推):**  $\partial L / \partial \hat{y} = -2(1 - \hat{y}) = -3$ ;  $\partial L / \partial W_2 = -3 \cdot a_1 = [-3, -6]$  (注意这里是损失对  $W_2$  的梯度, 等于"上游梯度  $\times$  本层输入");  $\partial L / \partial a_1 = -3 \cdot W_2^T = [-3, 6]$  (误差往隐藏层投影); 由于  $a_1 = z_1$  都正, ReLU 导数为 1,  $\delta_1 = [-3, 6]$  不变;  $\partial L / \partial W_1 = \delta_1 \cdot x^T = [-3, 6]^T \cdot [1, 2] = \begin{bmatrix} -3 & -6 \\ 6 & 12 \end{bmatrix}$ 。用  $\eta=0.1$  跑一次梯度下降, 所有参数就更新了一步。

## 7.10 应用实例：MNIST 手写识别

MNIST 是  $28 \times 28$  灰度手写数字 0-9 的标准数据集，共 6 万训练 + 1 万测试样本。它是深度学习的"hello world"——你的网络结构、训练流程、调试技巧都能在这个数据集上快速验证。

一个 [784-128-64-10] 的 MLP，ReLU 激活、交叉熵损失、Adam 优化器，训练 20 轮就能在测试集达到 98% 准确率。换成简单 CNN 能突破 99%，更复杂的 Vision Transformer 能到 99.7%。对比一下：上一章 SVM 在 MNIST 上是 98.5%——已经很不错了，但 MLP/CNN 不需要仔细调核函数，靠堆深度就能稳定提升。这就是神经网络在大数据上的优势：模型越深，能力越强，且不需要手工设计特征。

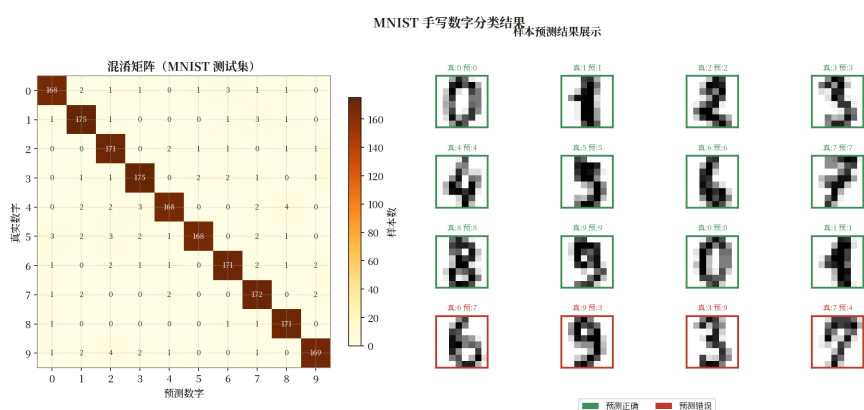


图 7.3 MNIST 训练曲线与测试集部分预测结果

## 7.11 代码实现：PyTorch MLP

现代深度学习都用框架写，从零手写反向传播既繁琐又容易出错。下面是 PyTorch 版本的 MLP，注意它怎么把"前向 + 反向 + 优化"三步写出来：

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = 'cuda' if torch.cuda.is_available() else 'cpu'
tf = transforms.ToTensor()
train_ds = datasets.MNIST('.', train=True, download=True, transform=tf)
test_ds = datasets.MNIST('.', train=False, download=True, transform=tf)
tr = DataLoader(train_ds, batch_size=128, shuffle=True)
te = DataLoader(test_ds, batch_size=256)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(64, 10))
    def forward(self, x): return self.net(x)

model = MLP().to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(10):
    model.train()
    for x, y in tr:
        x, y = x.to(device), y.to(device)
        opt.zero_grad()

        out = model(x)
        loss = loss_fn(out, y)
        loss.backward()
        opt.step()
    # eval
    model.eval(); correct = 0
    with torch.no_grad():
        for x, y in te:
            x, y = x.to(device), y.to(device)
            pred = model(x).argmax(1)
            correct += (pred == y).sum().item()
    print(f'epoch {epoch}, test acc = {correct/len(test_ds):.4f}')
```

## 7.12 现代发展与小结

从 MLP 出发，深度学习衍生出众多结构。**CNN**（卷积网络）利用局部连接与权值共享处理图像，代表作 ResNet、EfficientNet；**RNN/LSTM** 处理序列，曾统治机器翻译；**Transformer** 用自注意力替代循环，并行性强、表达力高，催生了 BERT、GPT 系列；**大语言模型 LLM**（GPT-4、Claude、Gemini）在千亿参数规模上展现出涌现能力，推动 AI 进入通用化时代。

但万变不离其宗——本章讲的**前向传播、反向传播、链式法则、激活函数、损失、优化器、正则**，是所有现代深度网络的地基。CNN 改的是连接方式，Transformer 改的是注意力机制，但训练流程依然是“前向算预测 → 算损失 → 反向求梯度 → 优化器更新参数”。掌握本章，你就掌握了深度学习的核心。

**[技巧]** 神经网络调参三件套：(1) 学习率是首要，先  $1e-3$  试 Adam，不行调到  $1e-4$  /  $3e-4$ ；(2) 过拟合加 Dropout/正则/数据增强，欠拟合加宽加深或延长训练；(3) 梯度爆炸/消失用 BatchNorm、残差连接、梯度裁剪。



## 第八章 降维与特征工程

到这里你可能已经发现：前面所有算法的输入都是**特征向量**。线性回归、SVM、神经网络，都假设你已经把数据整理成了一组数字。但真实业务里，特征的来源五花八门：一张  $224 \times 224$  的图像拉平就是 50176 维；一段文本用 TF-IDF 表示动辄几万维；一个用户的行为序列建模出来也有上千维。

维度一多就出问题。首先是**维度灾难**：高维空间里样本变得极稀疏，原来"近邻"这种直觉失效——所有点之间的距离都差不多，KNN 这种基于距离的算法基本废掉。其次是**计算爆炸**：1000 维数据训练 SVM 或神经网络，时间和内存都吃不消。更隐蔽的问题是**冗余**：1000 个特征里，可能 800 个高度相关，200 个纯噪声，真正有用的只有几十个——把这一堆全塞给模型，反而让它学不准。

所以这一章我们解决两个相关的问题：**降维**——把高维数据压到低维，保留本质、去掉冗余；**特征工程**——把原始数据加工成模型能用好用的特征。这两件事一起决定了下游模型能达到的上限。

### 8.1 PCA 的直觉：找数据变化最大的方向

最经典的降维方法是**主成分分析**（Principal Component Analysis, PCA）。我们先不讲公式，先想一个问题：假如 1000 维数据其实主要就沿着一个方向变化（其他 999 个方向上的取值都差不多），那你只要保留这 1 个方向上的投影，几乎没丢信息。反过来说：**数据变化最大的方向，就是信息最丰富的方向**。

为什么这么说？因为"变化大 = 信息多"。如果某个方向上所有样本的取值都一样（方差为 0），那这个方向完全没区分度——你保留它也没用，反正谁都知道取值是这个常数。反方向看，方差越大的方向上，样本越分散，区分度越高，信息越丰富。所以 PCA 的核心想法就一句话：**找方差最大的方向，把它当作第一主成分**。

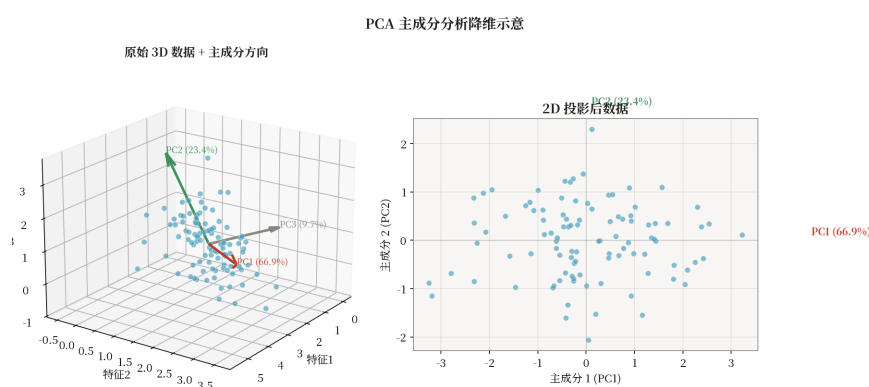


图 8.1 PCA 在 2D 上的示意：第一主成分沿方差最大方向，第二主成分与之正交

找完第一主成分，再找第二主成分——但第二主成分要和第一主成分**正交**（垂直），否则就和第一主成分高度重复了。在"和第一主成分正交"的约束下，再选方差最大的方向，就是第二主成分。以此类推，可以找出  $d$  个互相正交的主成分，按方差从大到小排序。如果只保留前  $k$  个，就把  $d$  维数据降到了  $k$  维——丢掉的是方差最小的那些方向，损失最小。

## 8.2 PCA 的数学推导：协方差矩阵与特征值分解

直觉有了，接下来一步步把它写成数学。先做**中心化**：把所有样本减去均值向量  $\mu$ ，让数据以原点为中心——这样后续计算方差就只需要算“样本点自身的内积平均”，不用再减均值。中心化后的数据矩阵记作  $X \in \mathbb{R}^{n \times d}$ ，每行是一个样本。

**第一步，算协方差矩阵。**方差公式  $\text{Var}(x) = (1/n)\sum (x_i - \mu)^2$ ，推广到多维就是协方差矩阵  $C = (1/n) \cdot X^T X$ ，它是个  $d \times d$  的对称矩阵，对角元素是每个特征的方差，非对角元素是特征间的协方差。C 完整描述了“数据各方向上的变化情况”。

**第二步，做特征值分解。**对称矩阵有个非常好的性质：可以写成  $C = W \Lambda W^T$ ，其中 W 是特征向量组成的正交矩阵， $\Lambda$  是特征值组成的对角矩阵。这里关键的一步是：**特征向量就是“主成分方向”，特征值就是“该方向的方差”**。所以把特征值从大到小排序，前 k 个特征值对应的特征向量就是前 k 个主成分。

**第三步，投影。**把前 k 个特征向量组成投影矩阵  $W_k \in \mathbb{R}^{d \times k}$ ，降维结果就是  $Z = X \cdot W_k$ 。总结一下完整公式：

$$C = (1/n) X^T X = W \Lambda W^T, Z = X \cdot W_k$$

工程实现上有个小技巧：直接对 C 做特征值分解复杂度是  $O(d^3)$ ，当 d 很大时（比如图像特征）非常慢。替代方案是用**奇异值分解**（SVD）： $X = U \Sigma V^T$ ，V 的前 k 列正好就是  $W_k$ ，且 SVD 在数值上更稳定、对稀疏矩阵也友好。所以 scikit-learn 的 PCA 实现用的是 SVD，不是显式特征值分解。

## 8.3 算法实现描述：PCA 的伪代码

把上面三步串起来，就是 PCA 的完整算法：

### 8.3.1 数据结构

- **mean\_**: 向量 [d]，训练集均值（用于中心化）
- **components\_**: 矩阵 [k, d]，前 k 个主成分（特征向量）
- **explained\_variance\_**: 向量 [k]，对应特征值
- **explained\_variance\_ratio\_**: 向量 [k]，方差解释比例

### 8.3.2 fit(X) — 协方差分解

输入:  $X \in \mathbb{R}^{n \times d}$ ，目标维度 k

步骤:

1. **中心化**:  $\mu \leftarrow \text{mean}(X, \text{axis}=0)$ ,  $X \leftarrow X - \mu$
2. **协方差矩阵**:  $C = (1/n) \cdot X^T X$  ([d, d] 对称矩阵)
3. **特征值分解**（或 SVD）: 求解  $C \cdot v = \lambda \cdot v$
4. **排序**: 按  $\lambda$  降序排列特征值和特征向量
5. **取前 k 个**:  $\text{components\_} = [v_1, v_2, \dots, v_k]^T$

- 6.  $\text{explained\_variance\_} = [\lambda_1, \dots, \lambda_k]$
- 7.  $\text{explained\_variance\_ratio\_} = \lambda_1 / \sum \lambda$

### 8.3.3 transform(X) 与 inverse\_transform

$$Z = (X - \mu) \cdot \text{components\_}^T \in \mathbb{R}^{n \times k}$$

**inverse\_transform** (有损重建) :  $X = Z \cdot \text{components\_} + \mu$ 。仅当  $k = d$  时可完美重建;  $k < d$  时有信息损失。

### 8.3.4 复杂度

- **训练**:  $O(d^3)$  协方差矩阵特征分解 (SVD 实现更快)
- **投影**:  $O(n \cdot d \cdot k)$  矩阵乘法

**[提示]** scikit-learn 用 SVD 分解替代协方差特征分解, 数值更稳定且避免  $O(d^3)$ 。

## 8.4 计算示例: 2D $\rightarrow$ 1D PCA 手算

来手算一次。考虑 3 个 2D 样本  $X = [[1,1],[3,3],[5,5]]$ ——它们正好在  $y = x$  直线上。

**第一步, 中心化**: 均值  $\mu = [(1+3+5)/3, (1+3+5)/3] = [3, 3]$ ,  $X\_c = [[-2,-2],[0,0],[2,2]]$ 。

**第二步, 协方差矩阵**:  $C = (1/3) \cdot X\_c^T X\_c = (1/3) \cdot [[8,8],[8,8]] = [[2.67, 2.67],[2.67, 2.67]]$ 。

**第三步, 特征值分解**:  $C$  的特征值是  $\lambda_1 = 5.33$  (对应特征向量  $w_1 = [1,1]/\sqrt{2}$ , 即数据变化最大的方向——正是  $y = x$  这条线) 和  $\lambda_2 = 0$  (对应  $[1,-1]/\sqrt{2}$ , 这个方向上数据完全没变化)。

**第四步, 取最大主成分  $w_1$ , 投影到 1D**:  $z = X\_c \cdot w_1 = [-2\sqrt{2}, 0, 2\sqrt{2}] \approx [-2.83, 0, 2.83]$ 。原来的二维数据被压到一维, 但样本的**相对顺序关系**完整保留。这是 PCA 的本质: 在方差最小的方向上"塌缩"维度, 保留信息最多的方向。

## 8.5 t-SNE: 当 PCA 找不到非线性结构时

PCA 有个根本局限——它是**线性**的。它只能找直线方向, 对弯曲的、卷曲的、嵌套的非线性结构完全没办法。比如著名的"瑞士卷"数据: 三维空间里一张卷起来的曲面, PCA 会找到三个垂直的主轴, 但任何一个轴都无法还原"卷起来的"二维结构。图像、文本这类真实数据的结构通常是非线性的, 所以光靠 PCA 不够。

怎么解决? **t-SNE** (t-distributed Stochastic Neighbor Embedding, Maaten & Hinton 2008) 的思路完全不同: 它不再找方向, 而是在**高维空间算样本间的相似度**, 然后在**低维空间尽量保持这种相似度**。具体来说分两步。

**第一步, 在高维空间用高斯核度量相似度**:  $p_{j|i} \propto \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$ , 表示"在  $x_i$  附近  $x_j$  出现的概率"。 $\sigma_i$  由困惑度参数控制。**第二步, 在低维 (通常 2D) 用 t 分布核度量相似度**:  $q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$ 。注意低维用的是**重尾**的 t 分布 (自由度 1 的 Student-t), 不是高斯——这个选择很关键: 它能缓解"拥挤问题", 把高维中距离中等、本来在低维会被挤在一起的样本"推开"。

然后最小化两个相似度分布之间的**KL 散度**：

$$L = \sum_i \text{KL}(P_i \parallel Q_i) = \sum_{i,j} p_{ij} \log(p_{ij}/q_{ij})$$

用梯度下降优化低维坐标  $y_i$ ，最终得到的 2D 散点图能非常漂亮地把高维聚类结构展示出来。t-SNE 在 MNIST 上的可视化效果惊艳——10 个数字会形成 10 个清晰的簇，即使 PCA 把它降到 50 维也看不到这么清楚的结构。

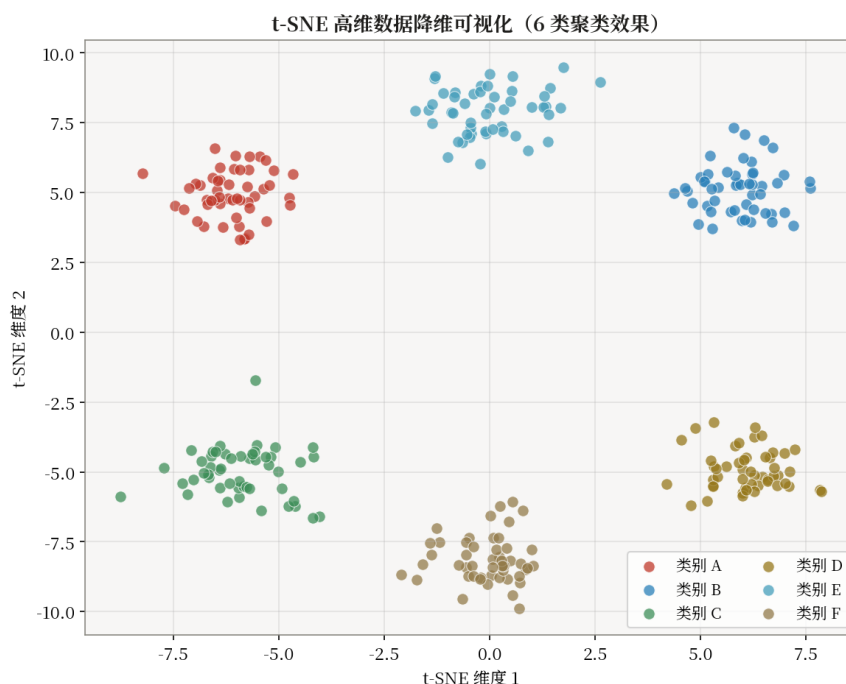


图 8.2 t-SNE 在 MNIST 上的可视化：不同数字在 2D 形成清晰簇

但 t-SNE 也有代价：计算复杂度  $O(n^2)$ ，几万样本就要跑几十分钟；而且每次结果可能略有不同，不能直接作为下游特征。**UMAP** (2018) 基于拓扑学，效果与 t-SNE 相当但速度快一个量级，还能保留更多全局结构，可以直接用于聚类或监督任务下游——所以近年 UMAP 在很多场景下替代了 t-SNE。

## 8.6 特征工程：再好的算法也救不了烂数据

降维解决的是"维度太多"，但还有一类问题降维解决不了——**数据本身不好**。机器学习圈有一条铁律叫 "Garbage in, garbage out"——喂进去的是垃圾，吐出来的也只能是垃圾。所以训练模型之前，必须把原始数据加工成"模型能用、好用"的特征。这一步叫**特征工程**，老练的工程师都知道：特征工程的天花板，往往就决定了模型的天花板。

特征工程包含几大块，我们一块块看。

**第一块，缺失值处理：填还是删？** 现实数据里缺失值很常见——用户没填年龄、传感器掉线、爬虫漏字段。简单做法是直接删掉含缺失的行，但如果缺失太多就删光了。更通用的做法是**填充**：数值特征用均值或中位数填，类别特征用众数填；更精细的做法是按相似样本填（KNN 填充）或用模型预测填（用其他特征当 X 训练一个回归器预测缺失）。注意：某些"是否缺失"本身就有信号（比如没填年龄可能是新用户），可以加一个"is\_missing"标志列当新特征。

**第二块，类别特征编码：怎么把类别变成数字？**模型只认数字，但很多特征是文字——"北京/上海/广州"、"男性/女性"、"高/中/低"。怎么编码？低基数类别（取值少）用**one-hot**：每个取值对应一个 0/1 列，简单但维度会膨胀；高基数类别（如城市、商品 ID）one-hot 会爆维，改用**target encoding**——把每个类别替换成"该类别下目标变量的均值"，比如把"北京"替换成"北京用户的违约率"。注意 target encoding 容易目标泄漏，要用交叉验证或平滑处理；CatBoost 自带 Ordered TS 自动处理这个问题，所以特别适合类别多的情况。

**第三块，数值变换：让分布更好看**有些特征分布严重长尾——比如收入、点击量——一个极端值就能把模型带偏。常用做法是**对数变换**  $\log(1+x)$ ，把长尾压成正态分布；或者**标准化**（z-score，减均值除标准差）让不同尺度的特征在同一量级；**归一化**（min-max 缩放到 [0,1]）适合已知取值范围的场景。这些变换看似简单，但对 SVM、神经网络这类对尺度敏感的模型影响很大。

**第四块，特征构造与选择**特征构造是把已有特征加工出新特征：交叉特征（ $A \times B$  捕捉交互效应）、统计聚合（按用户聚合最近 7 天点击数）、时间窗特征（"距上次购买天数"）。特征选择是反过来——剔除没用的：过滤法看特征与目标的相关系数或互信息；包裹法用模型递归选特征（RFE）；嵌入法则利用模型自带的特征重要性（Lasso 的 L1、树模型的 gain）。少而精的特征不仅训练快，还往往泛化更好。

## 8.7 应用实例：高维数据可视化

在生物医药单细胞测序中，单细胞表达矩阵常有 2 万基因  $\times$  数万细胞。直接看是不可能看的，必须降维。标准流程是：先做标准化与对数变换，PCA 降到 50 维保留 80%+ 方差，再用 UMAP/t-SNE 进一步降到 2D 可视化。不同细胞类型在 2D 图上形成清晰簇，便于发现新亚群、识别稀有细胞。同样流程也广泛用于金融客户分群、推荐系统用户嵌入分析、文本聚类场景。

## 8.8 代码实现：sklearn PCA 与 t-SNE

scikit-learn 的 PCA 和 TSNE 接口都很简单。注意 t-SNE 计算慢，通常先 PCA 降到 30-50 维再喂给 t-SNE：

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True) # 1797 x 64
X = StandardScaler().fit_transform(X)

# (1) PCA: 观察保留方差比
pca = PCA(n_components=30)
Z_pca = pca.fit_transform(X)
print(f'cumulative variance: {pca.explained_variance_ratio_.cumsum()[-1]:.3f}')

# (2) t-SNE: 降到 2D 可视化 (init=PCA 加速)
tsne = TSNE(n_components=2, init='pca', learning_rate='auto',
            perplexity=30, random_state=42)
Z_tsne = tsne.fit_transform(Z_pca) # 先 PCA 再 t-SNE, 速度更快

plt.figure(figsize=(6,5))
scatter = plt.scatter(Z_tsne[:,0], Z_tsne[:,1], c=y, s=8, cmap='tab10')
plt.legend(*scatter.legend_elements(), title='digit')
plt.title('t-SNE on digits (PCA→30D→t-SNE→2D)')
plt.tight_layout()
plt.savefig('digits_tsne.png', dpi=150)
```

## 8.9 现代发展与小结

降维与特征工程正在从"手工"走向"自动"。**自编码器** (Autoencoder) 用神经网络学习非线性瓶颈表示, 可以处理图像、序列等复杂结构; **变分自编码器** (VAE) 进一步引入概率结构, 可生成新样本; **对比学习** (Contrastive Learning, 如 SimCLR、CLIP) 通过对比正负样本对学习到强大的表示, 在下游任务上达到甚至超过有监督方法; **大模型特征** (LLM embedding) 将文本、图像统一编码到高维语义空间, 成为推荐、检索、RAG 的基础设施。

但万变不离其宗——PCA 给的"找最大方差方向"、t-SNE 给的"保持相似度"、特征工程给的"把原始数据加工成模型能用的形式", 是所有现代特征学习的核心思想。掌握本章, 你就拥有了从原始数据到模型输入的完整加工链路。

**[注意]** 常见陷阱: t-SNE 不能用于下游监督任务, 它仅适合可视化且每次结果不同; 若需稳定降维供下游使用, 优先 PCA 或 UMAP。PCA 前必须中心化与标准化, 否则尺度大的特征会主导主成分。



## 附录 A 数学基础

看到这你可能会发现：前面几章反复出现一些相同的"工具"——向量、矩阵、特征值、概率分布、梯度、链式法则、熵。它们不是为某个算法服务的，而是贯穿所有算法的"通用语言"。

与其每次都临时查，不如把它们集中讲清楚一次。但这个附录不想写成本数学手册——我们用"为什么需要它"的视角介绍：每一块数学工具，**对应了前面哪个算法的哪一步**。把它们当成理解前面算法的钥匙，而不是孤立的公式。

### A.1 线性代数：向量是数据，矩阵是变换，特征值是主方向

**向量是有序的数字序列**  $x = [x_1, \dots, x_d] \in \mathbb{R}^d$ 。在机器学习里，向量几乎就等于"一条数据"——一个用户的所有特征拼起来是一个向量，一张图像拉平后是一个向量。两个向量的**内积**  $x \cdot y = \sum x_i y_i$  度量相似度（SVM 的核函数本质就是内积），向量**范数**  $\|x\|_p = (\sum |x_i|^p)^{1/p}$  度量长度（正则化里的 L1、L2）。

**矩阵是线性变换**。一个矩阵  $A \in \mathbb{R}^{m \times n}$  把向量  $x \in \mathbb{R}^n$  映射到  $y = A \cdot x \in \mathbb{R}^m$ 。神经网络每一层的本质就是矩阵乘法  $W \cdot x + b$ —— $W$  是"上一层特征到这一层特征的变换规则"。若  $A$  为方阵且行列式非零，则存在逆  $A^{-1}$  使  $A \cdot A^{-1} = I$ ，线性回归的闭式解就用逆矩阵  $(X^T X)^{-1} X^T y$ 。

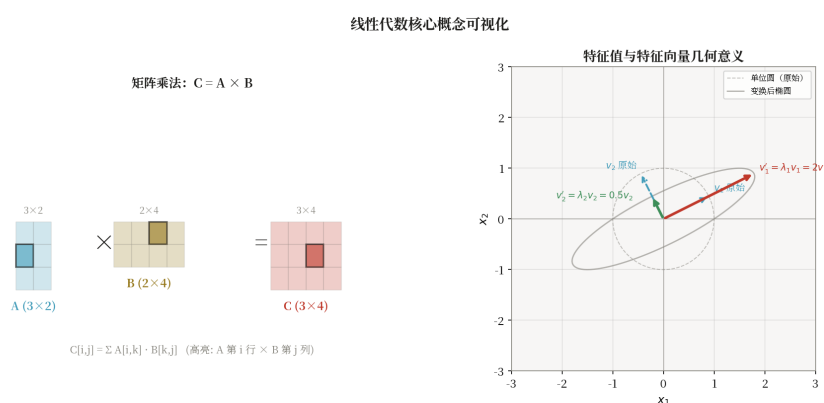


图 A.1 线性代数基本对象：向量、矩阵、特征值分解与几何解释

**特征值与特征向量是"主方向"**。对方阵  $A$ ，若存在标量  $\lambda$  与非零向量  $v$  使  $A \cdot v = \lambda \cdot v$ ，称  $\lambda$  为**特征值**、 $v$  为**特征向量**。直观理解： $A$  这个变换作用在  $v$  上时， $v$  的方向不变，只被  $\lambda$  倍拉伸。所以特征向量就是"变换后方向不变的轴"，特征值就是"沿这个轴的伸缩比例"。回到第八章 PCA：协方差矩阵的特征向量就是数据变化最大的方向（主成分），特征值就是该方向的方差——这就是为什么 PCA 用特征值分解。

对称矩阵的特征值都是实数、特征向量正交，可以写成  $A = Q \Lambda Q^T$ ——PCA、SVD 都依赖这一性质。**奇异值分解** SVD 推广到非方阵： $A = U \Sigma V^T$ ， $\Sigma$  的对角元素  $\sigma_i$  称奇异值。SVD 不仅能算 PCA，还是推荐系统协同过滤、自然语言处理 LSA 的核心工具。

### A.2 概率论：贝叶斯是"用新证据更新旧信念"

机器学习处理的是不确定性——预测总有误差，模型总有置信度。随机变量  $X$  的分布由概率质量函数（离散）或密度函数  $f(x)$ （连续）刻画。**期望**  $E[X] = \sum x \cdot p(x)$  或  $\int x \cdot f(x) dx$  反映

均值；方差  $\text{Var}(X) = E[(X-E[X])^2]$  反映离散度。两个随机变量独立当且仅当联合分布等于边缘分布乘积。

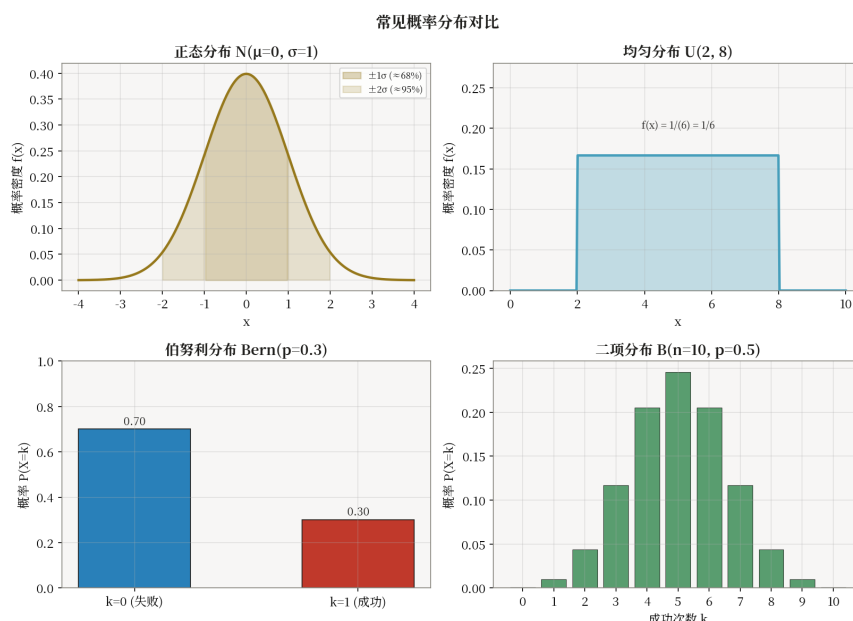


图 A.2 常见分布的概率密度函数：高斯、均匀、指数、伯努利

这一节最重要的一个公式是**贝叶斯定理**。它的核心思想用一句话就能讲明白：**用新证据更新旧信念**。在看到数据之前，你对参数有个"先验"判断  $P(\theta)$ ；看到数据  $D$  之后，你用似然  $P(D|\theta)$  修正它，得到"后验"  $P(\theta|D)$ 。更新公式就是：

$$P(\theta|D) = P(D|\theta) \cdot P(\theta) / P(D)$$

其中  $P(\theta)$  是先验（看你以前信什么）， $P(D|\theta)$  是似然（数据在你假设下的可能性）， $P(\theta|D)$  是后验（看完数据后的新信念）。这个公式是几乎所有概率机器学习方法的根：朴素贝叶斯分类器直接套用贝叶斯定理；最大似然估计（MLE）取  $\theta = \arg\max P(D|\theta)$ （不考虑先验）；最大后验估计（MAP）取  $\theta = \arg\max P(D|\theta) \cdot P(\theta)$ （带先验），等价于带正则的 MLE——所以 L2 正则可以理解为"假设参数服从高斯先验"，L1 正则可以理解为"假设参数服从拉普拉斯先验"。

常见的分布要熟悉：高斯  $N(\mu, \sigma^2)$ （噪声假设）、伯努利  $\text{Bern}(p)$ （二分类）、分类  $\text{Categorical}(p_1, \dots, p_k)$ （多分类）、泊松  $\text{Poisson}(\lambda)$ （计数数据）、指数  $\text{Exp}(\lambda)$ （等待时间）、Beta、Dirichlet（贝叶斯参数估计的共轭先验）。

## A.3 微积分：梯度是"变化最快的方向"

所有机器学习算法的训练，本质上都是在**最小化损失**。最小化损失的核心工具就是**梯度**。梯度  $\nabla f$  是多元函数对各变量偏导组成的向量，它的几何意义非常清楚：**指向函数增长最快的方向**。反过来，**负梯度  $-\nabla f$  就是下降最快的方向**——所以梯度下降就是"沿着负梯度走一小步"： $\theta \leftarrow \theta - \eta \cdot \nabla f$ 。从线性回归到神经网络，所有基于梯度的训练方法都遵循这个套路。

**链式法则是反向传播的数学基础**。若  $z = f(g(x))$ ，则  $dz/dx = (df/dg) \cdot (dg/dx)$ ；多元情形下用雅可比矩阵乘法表达。第七章反向传播的  $\delta_l = (W_{l+1}^T \cdot \delta_{l+1}) \odot \sigma'_l(z_l)$  本质上就是链式法则

——把"输出层误差"通过  $W_{l+1}$  "倒着投影"到本层，再乘上本层激活的导数。

**泰勒展开**把函数在某点附近用多项式逼近：

$$f(x) \approx f(a) + f'(a)(x-a) + f''(a)/2!(x-a)^2 + \dots$$

一阶泰勒展开是梯度下降的理论依据——在当前点附近，损失函数可以近似为线性，所以沿负梯度方向走能减小损失。二阶泰勒展开是牛顿法的基础——它不仅看梯度还看曲率（海森矩阵），更新更精确： $\theta \leftarrow \theta - H^{-1} \cdot \nabla f$ 。XGBoost 的"二阶展开"也是这个思路——用一阶  $g$  和二阶  $h$  两个量更精确地估计损失下降，分裂更准。海森矩阵  $H$  是二阶偏导矩阵，描述函数曲率，牛顿法收敛快但  $H$  在深度网络中代价过高，所以深度学习几乎都用一阶方法（SGD、Adam）。

## A.4 信息论：交叉熵是分类损失的"标配"

信息论是 Shannon 在 1948 年为通信理论创立的，但它对机器学习的影响深远——尤其是分类损失。先看**信息熵**： $H(P) = -\sum p_i \log p_i$  度量分布的不确定性。均匀分布熵最大（最不确定），确定性分布（某事件概率为 1）熵为 0（最确定）。决策树第二章里讲过的"信息增益"，本质就是"分裂前后熵的减少量"。

**交叉熵**  $H(P, Q) = -\sum p_i \log q_i$  度量"用分布  $Q$  编码真实分布  $P$  所需的平均比特数"。它是分类损失的标准选择——第七章神经网络的 Softmax + 交叉熵损失就是这么来的：让模型预测分布  $Q$  尽量接近真实标签分布  $P$  (one-hot)，就是最小化交叉熵。

$$H(P, Q) = -\sum_i p_i \log q_i$$

**KL 散度**  $KL(P \parallel Q) = \sum p_i \log(p_i/q_i)$  度量两个分布的差异，非负且不对称。一个关键等式是：**交叉熵 = 熵(P) + KL(P  $\parallel$  Q)**。由于  $P$  是固定的真实分布，熵( $P$ ) 是常数，所以**最小化交叉熵等价于最小化 KL 散度**。进一步可以推出"最大似然估计 = 最小化交叉熵 = 最小化 KL"——这是一个深刻而优美的等价，把统计学和机器学习的两条主线统一起来了。

信息论还是理解变分推断、VAE、强化学习中策略梯度的钥匙。掌握这一节，你会发现后面遇到的很多"新损失函数"——从分类交叉熵到对比学习的 InfoNCE——本质上都是信息论框架下的不同变形。

## 附录 B 集成学习框架

前面第五章我们手写了随机森林和 GBDT 的伪代码，把原理讲清楚了。但实际项目里你不会真的从零写——生产环境用的是几个高度优化过的现成库：scikit-learn、XGBoost、LightGBM、CatBoost。它们都遵循相似的"训练-验证-早停-特征重要性" workflow，但在底层算法、类别特征处理、超参数体系与速度上各有侧重。

这个附录就把这四个主流框架对比一下，让你知道在什么场景下该用哪个、调参时该重点看什么。

### B.1 Scikit-learn：通用入门的首选

scikit-learn 是 Python 机器学习的通用入口，提供 RandomForest、GradientBoosting、HistGradientBoosting 等集成实现。它的最大优点是**接口统一、文档清晰、上手快**——所有模型都遵循 `.fit()` / `.predict()` / `.score()` 三件套，换模型几乎不改代码。

核心配套工具也有三件套：**Pipeline** 把预处理与模型串成一条流水线避免数据泄漏；**GridSearchCV** / **RandomizedSearchCV** 做超参搜索；**cross\_val\_score** 做 K 折交叉验证。**HistGradientBoosting** 是 sklearn 的 GBDT 现代化重写，用了直方图算法（和 LightGBM 同思路），速度接近 LightGBM，且原生支持缺失值。如果不想装额外依赖，sklearn 自带这个就够用了。

### B.2 XGBoost：Kaggle 表格任务的常胜将军

XGBoost 自 2014 年开源以来一直是 Kaggle 表格任务的常胜将军。第五章我们讲过它的核心创新：二阶泰勒展开（同时用一阶  $g$  和二阶  $h$ ）、显式正则项  $\gamma \cdot T + \frac{1}{2}\lambda \cdot ||w||^2$ 、列抽样、并行分裂查找、稀疏感知。这里把它的关键超参列一下，方便调参时对照：

- eta（学习率）：典型 0.01~0.3，配合早停；
- max\_depth：常用 3~8，越深越易过拟合；
- subsample / colsample\_bytree：行/列采样；
- lambda / alpha：L2/L1 正则；
- min\_child\_weight：控制叶节点最小样本权重和；
- early\_stopping\_rounds：监控验证集，自动选择最佳迭代数。

XGBoost 提供原生 DMatrix 接口与 sklearn 接口，前者更高效、支持 GPU 与外存训练。特征重要性输出包括 gain、weight、cover 三种，建议以 gain 为准——它衡量"该特征带来的平均损失下降"，最贴近"重要性"的直觉。

### B.3 LightGBM：大数据上的速度冠军

LightGBM（微软，2017）在算法层面做了多项优化，第五章我们提过它的两大招：**直方图算法**把连续特征离散化到 255 桶，分裂复杂度由  $O(n \cdot d)$  降到  $O(b \cdot d)$ ，同时直方图差分加速兄弟节点分裂；**叶子优先**（leaf-wise）生长找当前损失下降最大的叶节点分裂，比层优先（level-wise）更易过拟合但精度更高，常配合 max\_depth 或 num\_leaves 控制。

它还有两个针对大数据的优化：**GOSS**（基于梯度的单边采样）保留梯度大的样本，因为梯度大的样本对学习更有用；**EFB**（互斥特征捆绑）把稀疏互斥特征合并降低特征数。关键超参：num\_leaves（一般  $\ll 2^{\text{max\_depth}}$ ）、min\_data\_in\_leaf、feature\_fraction、bagging\_fraction、lambda\_l1/l2。LightGBM 在 1 千万级样本上可在分钟级完成训练，是大数据 GBDT 的首选。

## B.4 CatBoost：类别特征的原生支持者

CatBoost (Yandex, 2017) 的招牌是**类别特征处理**。第八章讲特征工程时我们提到，类别变量需要编码才能喂给模型，常用 target encoding 又容易目标泄漏。CatBoost 用 **Ordered Target Statistics** 解决这个问题：用历史样本的目标均值编码类别，从机制上避免传统 target encoding 的泄漏。

它还有两个独特设计：**对称树结构**——所有节点在同一层用相同特征分裂，减少过拟合且显著加速推理；**Ordered Boosting**——在每轮提升时用与当前样本无关的子树计算梯度，进一步缓解预测偏差。CatBoost API 简洁：传入 cat\_features 列索引即可自动处理类别变量，在含大量类别特征的电商、广告、风控场景中常优于 XGBoost。

## B.5 框架对比与选型经验

把四个框架放一起对比一下，方便你选型：

| 框架              | 训练速度 | 精度上限 | 类别特征      | GPU | 适用场景      |
|-----------------|------|------|-----------|-----|-----------|
| scikit-learn GB | 中等   | 中    | 需预处理      | 部分  | 小数据入门     |
| XGBoost         | 快    | 高    | 需 one-hot | 原生  | Kaggle、通用 |
| LightGBM        | 最快   | 高    | 部分支持      | 原生  | 大数据、稀疏    |
| CatBoost        | 中等   | 高    | 原生最强      | 原生  | 类别特征多     |

几条工程经验：(1) 任何 GBDT 任务先用 LightGBM 跑通基线；(2) 用 Optuna 或 Hyperopt 做贝叶斯调参比网格搜索更高效；(3) 对含 100+ 类别特征的数据改用 CatBoost；(4) 多模型 Stacking 时一般 LightGBM + XGBoost + CatBoost + RF 组合，再加一个 LR 作为元学习器即可。



# 附录 C PyTorch 与 Transformers

第七章我们手写了 MLP 的伪代码——前向、反向、链式法则都自己算。但实际做深度学习项目时，你不会真的从零写反向传播。现代深度学习都用框架：框架帮你自动求导、自动分配 GPU、自动并行计算，你只需要把"前向怎么算"定义清楚，剩下交给框架。

深度学习框架经历了 Theano → TensorFlow 1.x → PyTorch 的演进，当前学术界与业界的事实标准是**PyTorch**。在它之上，HuggingFace Transformers 提供数千个预训练模型与简洁 API，让"使用大模型"变得像调用普通函数一样简单。这个附录就介绍 PyTorch 基础与 Transformers 工作流，帮你从"手写伪代码"过渡到"调框架做项目"。

## C.1 PyTorch 基础：Tensor、autograd、Module 三件套

PyTorch 的核心抽象是**张量**（Tensor）——一个支持 GPU 与自动微分的多维数组，可以理解为 NumPy ndarray 的"超级版"。所有神经网络计算最终都落到张量运算上。

**autograd** 引擎是 PyTorch 的灵魂。它跟踪张量上的运算构建**计算图**，你调用一次 backward()，引擎就自动从损失出发反向求出所有参数的梯度——这正是第七章讲过的"反向传播"的自动化实现。你不用再手写链式法则，框架帮你算。

**nn.Module** 是模型基类，通过组合层（nn.Linear、nn.Conv2d、nn.ReLU）构建网络。一个完整的训练循环包括五步：前向（model(x)）→ 损失（loss\_fn(out, y)）→ 清梯度（opt.zero\_grad()）→ 反向（loss.backward()）→ 优化器更新（opt.step()）。这五步对应了第七章手写伪代码里的所有逻辑，只是把它们封装成了简洁的 API。

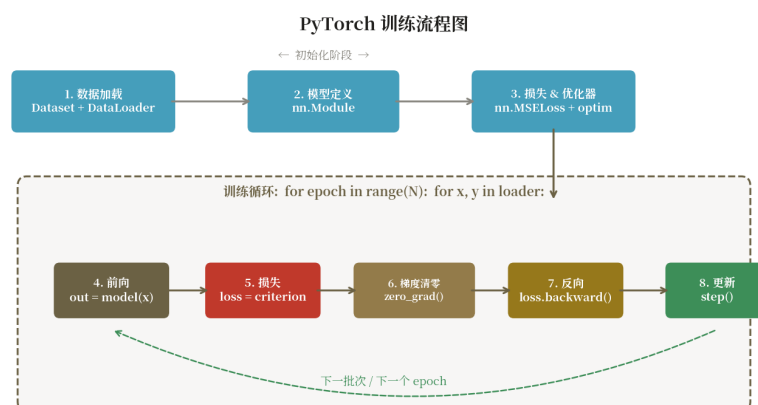


图 C.1 PyTorch 训练工作流：Dataset → DataLoader → Model → Loss → Optimizer → Loop

## C.2 数据加载：Dataset 与 DataLoader

训练循环之外，另一个重要组件是数据加载。**Dataset** 抽象"如何取一条样本"，需实现 `__len__` 与 `__getitem__` 两个方法——一个返回数据集大小，一个按索引返回一条样本。**DataLoader** 抽象"如何批量、乱序、并行加载"，参数 `batch_size` 控制 batch、`shuffle` 控制每 epoch 打乱、`num_workers` 控制多进程加载。



配合 `torchvision.transforms` 可以做图像增强（随机裁剪、旋转、颜色抖动），配合 `collate_fn` 可以处理变长序列（比如把不同长度的句子 padding 到同一长度）。在大数据训练时，数据加载往往是瓶颈，把 `num_workers` 设为 CPU 核心数能显著加速。

## C.3 HuggingFace Transformers: 让大模型触手可及

HuggingFace Transformers 是预训练模型的"中央仓库"，覆盖 BERT、GPT、T5、LLaMA、Qwen 等数千个模型。它把"加载预训练模型 + 微调 + 推理"封装成三件套：

- **pipeline**：一行代码搞定任务（情感分析、文本生成、问答、翻译…）。
- **AutoModel**：根据 checkpoint 名自动加载正确模型架构与权重。
- **AutoTokenizer**：自动加载对应分词器，把文本变成模型输入张量。

微调（Fine-tuning）流程也很标准：加载预训练模型与分词器 → 准备任务数据集 → 调用 Trainer 或手写训练循环 → 保存模型。Trainer 封装了混合精度、梯度累积、评估、保存、TensorBoard 日志等，是上手最快的微调方式。

## C.4 代码示例：BERT 文本分类微调

把上面的概念串起来，下面是一个完整的 BERT 文本分类微调代码：用 AutoTokenizer 处理文本，AutoModelForSequenceClassification 加载预训练 BERT，Trainer 完成训练循环。

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import load_dataset

ckpt = 'bert-base-chinese'
ds = load_dataset('csv', data_files={'train': 'train.csv', 'test': 'test.csv'})
tok = AutoTokenizer.from_pretrained(ckpt)

def tokenize(batch):
    return tok(batch['text'], padding='max_length', truncation=True, max_length=128)

ds = ds.map(tokenize, batched=True)
ds.set_format('torch', columns=['input_ids', 'attention_mask', 'label'])

model = AutoModelForSequenceClassification.from_pretrained(ckpt, num_labels=2)

args = TrainingArguments(
    output_dir='./bert-cls', num_train_epochs=3,
    per_device_train_batch_size=32, per_device_eval_batch_size=64,
    learning_rate=2e-5, warmup_ratio=0.1, weight_decay=0.01,
    eval_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True, fp16=True)

trainer = Trainer(model=model, args=args,
    train_dataset=ds['train'], eval_dataset=ds['test'])
trainer.train()
trainer.save_model('./bert-cls-best')

# 推理
from transformers import pipeline
```

```
clf = pipeline('text-classification', model='./bert-cls-best', tokenizer=ckpt)
print(clf('这部电影真的很棒! '))
```

## C.5 进阶话题与小结

当模型与数据规模变大时，单卡训练不够用，需要**分布式训练**：DDP（DistributedDataParallel）做数据并行，最简单也最常用；FSDP（Fully Sharded Data Parallel）与 DeepSpeed ZeRO 切分优化器状态、梯度与参数，可以训练数十亿至上百亿参数模型。**LoRA / QLoRA** 等参数高效微调（PEFT）方法只训练少量低秩适配器参数，让消费级 GPU 也能微调大模型；**量化**（INT8/INT4）把权重压缩到更低精度，推理速度与显存占用大幅改善。

LLM 时代的典型工作流是这样的：选基座模型（Llama-3、Qwen、GLM 等）→ 用 PEFT 在领域数据上做 SFT（监督微调）→ 可选做 DPO/RLHF 对齐 → 量化部署（vLLM / TensorRT-LLM / Ollama）服务上线。掌握本附录的 PyTorch 与 Transformers 工具链，是从“会调包”到“会做研发”的关键一步。

**[注意]** 实践中常见的踩坑点：(1) 标签列名务必叫 label/labels 否则 Trainer 报错；(2) BERT 系最大长度 512，超长需切分或换 Longformer；(3) fp16 训练损失变 NaN 多半是 LR 过大或梯度裁剪未加；(4) 推理前 model.eval() 与 torch.no\_grad() 缺一不可。